GROUP1 SPRINT / FOUNDATION READING

Computer systems and hardware

Science & Technology / English

SOURCE BOOK

Introduction to Computer Science

AUTHORS

OpenStax and its contributing authors

PUBLISHER

OpenStax, Rice University

LICENSE

CC BY-NC-SA 4.0

ORIGINAL PDF PAGES

203-248 (PDF page positions, inclusive)

SOURCE

https://openstax.org/details/books/introduction-computer-science

REUSE

Attribution retained; chapter pages reproduced as-is.

SCOPE

Foundation study, not APPSC PYQs or an India/AP-specific chapter.

CHECKED

12 October 2026

No publisher endorsement is implied. Review against the current official syllabus.

OpenStax
Rice University
6100 Main Street MS-375
Houston, Texas 77005

To learn more about OpenStax, visit https://openstax.org.
Individual print copies and bulk orders can be purchased through our website.

©2026 Rice University. Textbook content produced by OpenStax is licensed under a Creative Commons Attribution Non-Commercial ShareAlike 4.0 International License (CC BY-NC-SA 4.0). Under this license, any user of this textbook or the textbook contents herein can share, remix, and build upon the content for noncommercial purposes only. Any adaptations must be shared under the same type of license. In any case of sharing the original or adapted material, whether in whole or in part, the user must provide proper attribution as follows:

- If you noncommercially redistribute this textbook in a digital format (including but not limited to PDF and HTML), then you must retain on every page the following attribution:
  "Access for free at openstax.org."
- If you noncommercially redistribute this textbook in a print format, then you must include on every physical page the following attribution:
  "Access for free at openstax.org."
- If you noncommercially redistribute part of this textbook, then you must retain in every digital format page view (including but not limited to PDF and HTML) and on every physical printed page the following attribution:
  "Access for free at openstax.org."
- If you use this textbook as a bibliographic reference, please include https://openstax.org/details/books/introduction-computer-science in your citation.

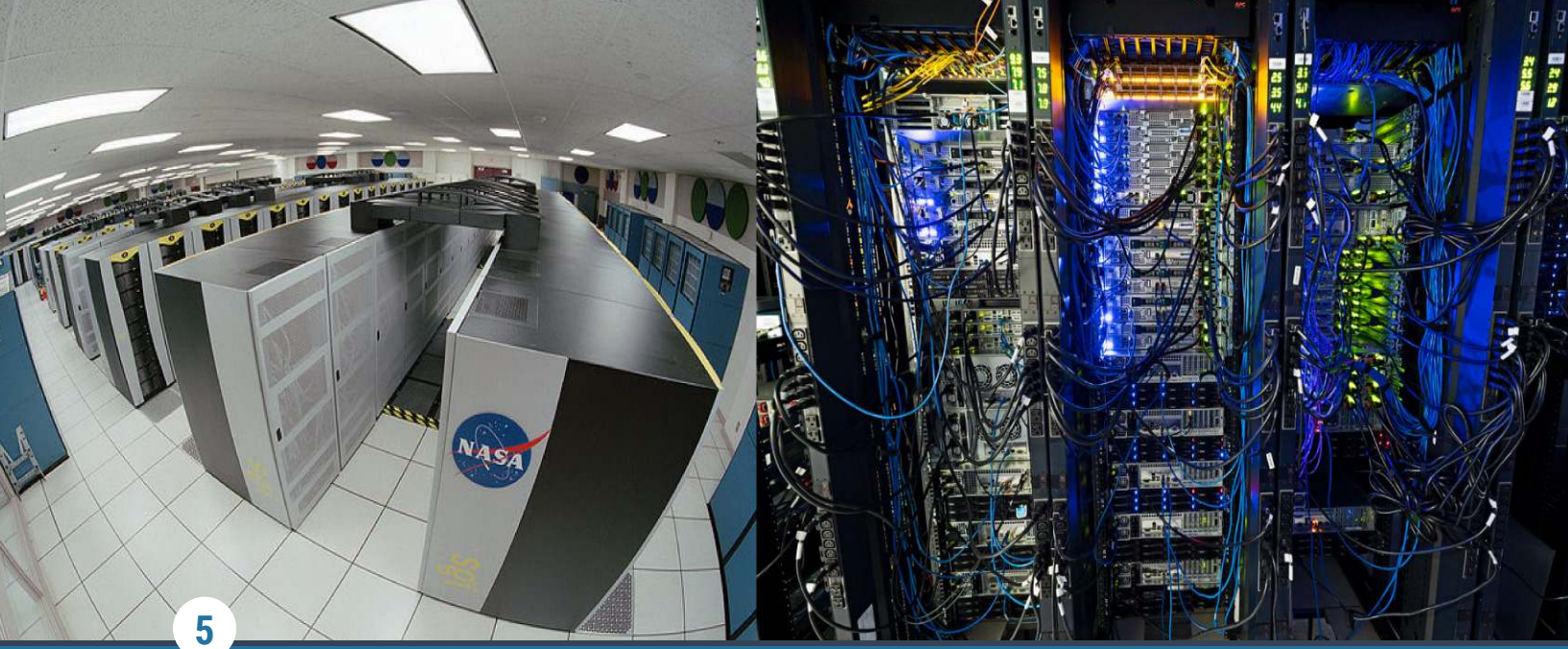
For questions regarding this licensing, please contact support@openstax.org.

Trademarks
The OpenStax name, OpenStax logo, OpenStax book covers, OpenStax CNX name, OpenStax CNX logo, OpenStax Tutor name, Openstax Tutor logo, Connexions name, Connexions logo, Rice University name, and Rice University logo are not subject to the license and may not be reproduced without the prior and express written consent of Rice University.

Kendall Hunt and the Kendall Hunt Logo are trademarks of Kendall Hunt. The Kendall Hunt mark is registered in the United States, Canada, and the European Union. These trademarks may not be used without the prior and express written consent of Kendall Hunt.

COLOR PAPERBACK BOOK ISBN-13	**978-1-711471-83-9**
B&W PAPERBACK BOOK ISBN-13	**978-1-711471-82-2**
DIGITAL VERSION ISBN-13	**978-1-961584-58-7**
ORIGINAL PUBLICATION YEAR	**2024**

2 3 4 5 6 7 8 9 10 CJP 26 24



5

Hardware Realizations of Algorithms: Computer Systems Design

Figure 5.1 What does the word *computer* really mean? For example, supercomputers (left) and data centers (right) are types of computers. "(credit left: modification of “Columbia Supercomputer—NASA Advanced Supercomputing Facility” by Trower, NASA/ Goddard Space Flight Center, Public Domain; credit right: modification of "2020 Data Center" by Jefferson Lab/Flickr, Public Domain)"

Chapter Outline

5.1 Computer Systems Organization
5.2 Computer Levels of Abstraction
5.3 Machine-Level Information Representation
5.4 Machine-Level Program Representation
5.5 Memory Hierarchy
5.6 Processor Architectures

Introduction

We use the word *computer* a lot, but we may not know a precise definition of it. More often than not, we use it to mean our desktops and laptops. But computers exist in many different forms, like your laptop, smartphone, or tablet. Embedded processors are used to power smart home security systems. When you access your social media accounts on Facebook, Twitter, Instagram, or any website, you are using very big computers hosted by these companies. These huge and powerful computers are clusters of computers hosted in data centers and supercomputers for some applications.

These computers have many things in common, but they also differ in many aspects. In general, computers all have processors, memory, storage, and input/output devices such as keyboards, screens, and speakers. What are these components? And how do they interact with each other to form what we know as a computer? This is what we will explore in this chapter. The difference between the computer inside your watch and the one running the big sites is the number and strength of processors (computer brain), the size of memory and disks (main means of storage), and how these many pieces are connected to each other. But the main concepts are the same.

A computer’s main job, as you may have guessed, is to do computations—a lot of computations. The faster a computer, the more computations it can do per second. All computer programs you use are based on computations, whether a modern immersive graphics intensive game that leverages artificial intelligence (AI),

a text editor, or a web browser. In this chapter, you will learn how computers can do computations in a fast and correct manner using processors, memory, disks, and other related hardware.

A company called TechWorks is taking advantage of leverages such as the latest nanotechnology, processor models known as neuromorphic processors, to enable the creation of the next generation of super society intelligent autonomous solutions (e.g., advanced robotics, autonomous cars and drones, or other autonomous systems). The use of Intel's Kapoho Point 8-chip Loihi 2 board technology allows TechWorks' developers to solve larger problems by stacking large-scale workloads and enabling AI models with up to one billion parameters and solving optimization problems with up to eight million variables.

5.1 Computer Systems Organization

Learning Objectives

By the end of this section, you will be able to:

- Define a computer system
- Explain how information is stored and transferred in a computer system
- Differentiate between high-level and machine-level programs
- Identify the elements of a typical computer system

At its core, a computer system is an electronic device that does computations. These computations appear to the outside world as executing programs. When you play a game, listen to a song, or browse the web, you are instructing your computer to do computations. You may wonder how does the computer function and how do the computations performed such as browsing the web or listening to a song relate to one another?

Let us start with the second part of that question: the relationship between computation and executing a program. A song that has been digitized for storage is actually a bunch of numbers that the computer reads; it produces sound based on those numbers. The computer must calculate (i.e., compute) the frequency and volume of the sound based on the numbers read. Another example is when you open a browser and type the address of a website. The computer reads what you typed on the keyboard and leverages network capabilities to translate it to a long number called an Internet Protocol (IP) address (Figure 5.2). The IP address consists of several digits, similar to your phone number, that allow the data to reach your computer over the Internet and for other computers over the Internet to recognize your computer. Your computer passes that IP address over the Internet to another big computer asking for the content of the required website, receives the content from that big computer, and executes browser software that translates it to content such as pictures, sounds, and animations on the screen. All these steps involve computations. But how does the computer do all this? To answer this question, we first need to know the components of a computer system.

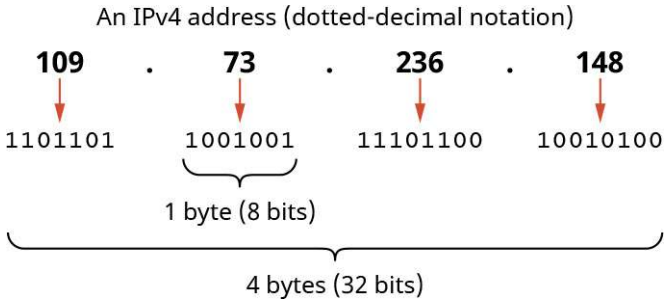


Figure 5.2 IP addresses use a standardized dotted-decimal notation that corresponds to a string of bits. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Computer Systems

A computer system consists of two major parts: hardware and software. The hardware includes the physical components of the computer system, such as hard drives, motherboards, and processor chips. The software consists of the programs you execute, such as media players or web browsers, and the data needed for these

programs to function. For example, the media player is a program, and the songs that you listen to using the media player are data. So, the hardware executes programs that are fed with data. Computers come in different shapes, from the small ones in your smartwatch, for example, to big supercomputers. Different computers may have different components besides the components that exist in all computer shapes. Bigger computers, datacenters, and supercomputers need extra components for getting data from multiple sources, more sophisticated parts to connect to the Internet, and cooling equipment. Mainframes that were created a few decades ago are big computers used for a different purpose. You don't need those in your smartphone.

Data Storage Inside a Computer

All programs and their corresponding data are stored inside a computer or downloaded from the Internet. So how are these programs and data stored? And how does a computer understand and execute them? Computers do not understand English or other natural languages; they can only relate to strings of 1s and 0s and therefore all the programs and data must be stored inside the computer as a sequence of 1s and 0s. A 1 or a 0 is called a binary digit or **bit**. Every 8 bits unit is called a **byte**. This is pretty much how computers that depend on electricity work. Other types, still not in the mainstream market but in design and testing stages, such as quantum computers, do not use bits but use something else called quantum bits or qubits.

If we say that a text editor program takes 1 megabyte of storage, it means that the editor is stored inside the computer, more specifically in the **disk** of your computer, which is a storage mechanism for data, as a sequence of almost 1 million bytes (1 megabyte is a bit larger than 1 million bytes). We call a program stored inside a computer an **executable**. Since programs and data are stored as a sequence of bits or bytes, what is the difference between a program and the data?

THINK IT THROUGH

Why Learn About Computer Systems?

Whether you want to be a software developer, a programmer, or a hardware designer, or even to efficiently use any computer system, you need a minimum knowledge of computer organization. You need to know what the different pieces in the computer systems are, how they interact, and how fast each piece is. Knowing the internal workings of computers helps you write more efficient software, design computers of different sizes (like the one in your watch, the one in your phone, or big supercomputers), and make the best decisions when buying a computer.

Application Programs and Executables

The programs we use in our computer systems, called applications, have been designed and written by software developers using computer languages, also called high-level languages (HLLs). HLLs have been designed to make the interaction between software developers and computers easier. However, computers do not understand HLLs. Computers understand only 1s and 0s. So, there is a set of programs designed to translate the programs written in an HLL into the 1s and 0s that computers can relate to. The programs written by software developers are translated into a set of instructions such as, "Add number 1 to number 2 and save the result as number 3." These instructions are stored as a series of 1s and 0s. Each group of those 1s and 0s represents a single instruction. All the instructions, in their representation as 1s and 0s, are stored in a file called an executable file on the disk.

When you click an icon or type a command, the operating system, software whose job is to manage the interaction between the user of the computer, the hardware, and the programs, loads the executable from storage into the computer memory. At this point, the computer is ready to start executing the program. Figure 5.3 shows the steps for writing programs, in various HLLs by software developers, and then translating these programs to an executable (the 1s and 0s representation of the program) using a special software toolchain

(Figure 5.3). This special software that translates HLLs to executables includes tools called compilers, assemblers, and linkers. We will not discuss these tools here, (refer to 5.2 Computer Levels of Abstraction and 5.4 Machine-Level Program Representation), but we now know what they do. Here, we will look at how the hardware executes instructions.

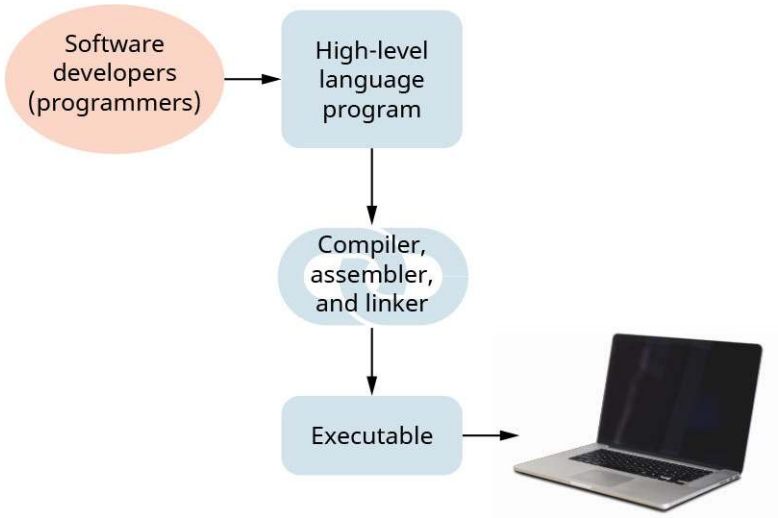


Figure 5.3 Programs developed in high-level languages (HLLs) such as C++, Java, or Python are then translated into executables that a computer can run. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Hardware Organization of Systems

Once the executable is in the memory, the hardware needs to do several things in order to execute it. Recall that an executable is a group of instructions in the form of 1s and 0s. The hardware first needs to fetch an instruction from memory by bringing it from the memory to inside the CPU, which is the brain of the computer that performs and executes the instructions of a program. Once the instruction is fetched from memory and stored in temporary storage inside the CPU, called a register, the CPU decodes this instruction; that is, it deciphers or understands the meaning of the several 1s and 0s that constitute the instruction. For example, the software program may contain an instruction such as "add this number x to the number y and put the result in z." Since computers do not understand English, this instruction is stored inside the computer in the form of 1s and 0s. The CPU has to read those 1s and 0s, understand what they mean (i.e., decode it), and then execute it.

After the instruction has been decoded, the CPU instructs the **arithmetic logic unit (ALU)** to execute it. The ALU is the piece of hardware inside the CPU that performs computations and logical operations such as comparisons. Once the instruction is executed, the result is saved into a register or sent back to the memory. The CPU is now ready to fetch the next instruction. Figure 5.4 shows a computer system's hardware. Inside the CPU we can see the ALU, a group of registers (called a **register file**), and a piece of hardware, called the memory controller, that helps the CPU talk to the memory. Inside the CPU we also find fast storage, which is faster than the memory but slower than the register, called cache memory. We will discuss cache memory in 5.5 Memory Hierarchy.

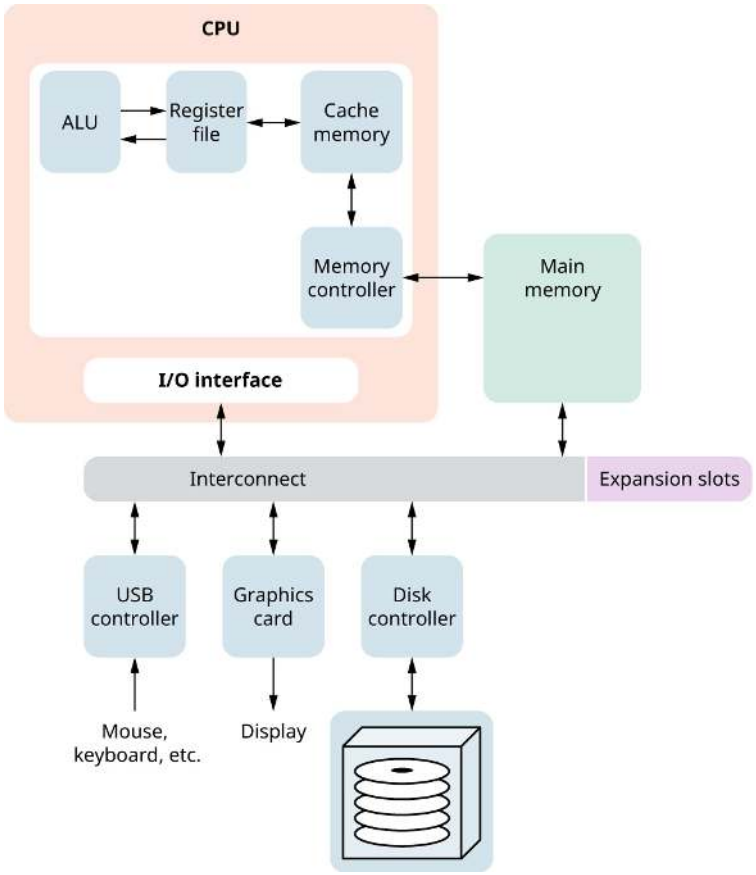


Figure 5.4 Any computer system consists of several components that, together, help the computer do its job of executing programs. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

One final piece of hardware inside the CPU is the I/O interface. The **input/output (I/O)** interface helps the CPU talk to the other I/O devices such as a keyboard or mouse. All the pieces—CPU, memory, graphics card—are connected via a collection of wires referred to as a bus. The purpose of a bus is to transfer data between the various pieces in the computer. Buses can differ in speed, which affects how fast the data are transferred. Figure 5.4 uses the generic word *interconnect* to designate buses. We can add more devices to the computer system such as a different display through expansion slots.

LINK TO LEARNING

Read this concise history of computers starting from the nineteenth century (https://openstax.org/r/76CompHistory19) to see how computers have changed and been developed over time.

Input/Output Devices

The job of input/output (I/O) devices is to take input from a user (i.e., typing on a keyboard or speaking into a microphone), transform it into 1s and 0s, and store this information in memory. It also takes some 1s and 0s, generated by some type of software program, from memory and translates them to an output format such as a picture on the screen or a sound from a speaker. Distinct devices have different speeds and varying ways of transforming the input and output to/from 1s and 0s.

Any I/O device, your keyboard for example, connects to the computer system using an interface such as a USB port that we see in all computers. The USB controller shown in Figure 5.4 is the piece of hardware that manages the USB port and allows it to detect that a device has been connected. Another important piece of hardware is the main memory.

Main Memory

For the CPU to execute programs and process data stored locally, it needs to obtain them from the computer's disk. But a disk is very slow. So, the CPU brings what it needs from the disk and temporarily stores it in faster storage referred to as main memory or random access memory (RAM). When you buy a laptop, one of the specifications is the amount of memory it has, such as 16 GB of RAM or 32 GB of RAM. The main memory is much faster than the disk and connected to the CPU with a faster interconnect, as shown in Figure 5.4. When you click an icon to start a program (e.g., your web browser), the program and its needed data are copied from the disk to the main memory. Then, the CPU reads the data and instructions from the memory into the CPU's registers and the ALU starts executing it.

Processor

The **processor**, also called the microprocessor, is another name for the CPU. It is the brain of the computer system, and its main job is to execute programs. As we discussed earlier, the processor fetches instructions from the memory, understands what each instruction wants to do, gets the data needed to execute the instruction, executes the instruction, and then stores the result in a register or in the main memory. It keeps doing so until the program ends. There have been huge advances in processor design leading to faster and more powerful computers. The processor in your smartphone today is more powerful than a big supercomputer was a few decades ago.

5.2 Computer Levels of Abstraction

Learning Objectives

By the end of this section, you will be able to:

- Describe abstraction levels from the highest to the lowest
- Explain application programs abstractions in relation to HLLs and instruction set architectures
- Discuss processor abstractions and how microarchitecture supports them
- Identify the role of the operating system within abstraction
- Discuss examples of new disruptive computer systems

When you look around, you see that complex systems can be viewed as layers of abstractions. The removal of unimportant elements of a program or computer code that distract from its process is called **abstraction**. This way of looking at complex systems makes it easier to understand them. For example, cars are very complicated inventions. At the highest level of abstraction, we look at a car as a set of devices used to operate it such as a steering wheel, brake and accelerator pedals, and so on. If we go to a lower level, we see devices that power the car such as its engine, gears, and spark plugs. If we take these parts and look at how they are designed, then we are at an even lower level where we see metals, plastics, and other materials. The same approach can be applied to computers to understand how they work. We can see computers as several layers of abstractions, as shown in Figure 5.5. For the remaining part of this section, we start from the highest level and then work through each abstraction layer to illustrate how it is used as a building block by the layer before it.

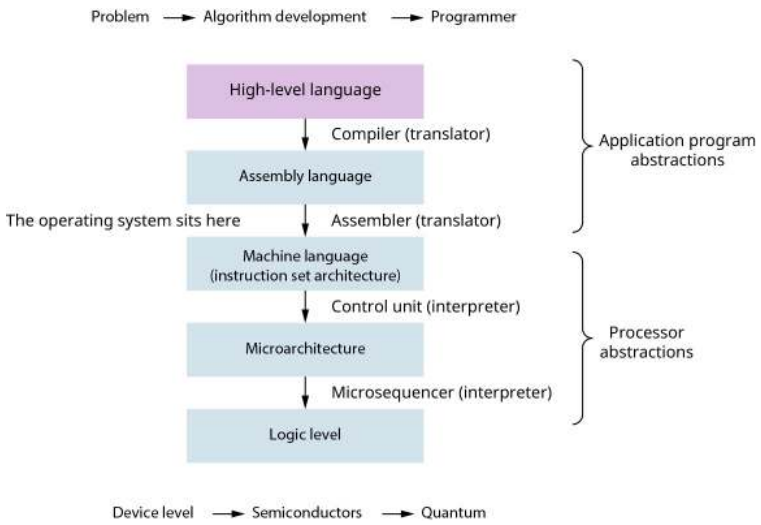


Figure 5.5 A computer system can be viewed at several levels of abstraction, each one layered on top of the other. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Computers are just a tool used to solve a problem. You may use computers to play games or listen to music, and in these cases, the problems that the computer is trying to solve are associated with programs that you use for entertainment purposes.

The top line in Figure 5.5 starts with the problem; we must have a very precise definition of the problem we are trying to solve with a computer. So, the first step in solving a problem with a computer is to know exactly, and with no vagueness, what we are trying to solve. You cannot make a computer solve a problem unless there is a defined and repeatable set of instructions to solve the problem. You may wonder then why it is necessary to use a computer in the first place if you can solve the problem yourself. Well, computers do not get bored, are precise, and can deal with very large problems. This is why, the next step after problem definition is to lay out the steps for solving the problem. This solution layout is called an algorithm. The algorithm is written in free format; that is, it can be steps written as a bulleted list, it can be a flowchart, or it can be a series of mathematical equations. Regardless of the format you choose for writing the algorithm, the algorithm needs to have a key set of characteristics.

The first characteristic is that an algorithm must be unambiguous. Each step of the algorithm must be very well defined and precise. The second characteristic is that the algorithm must be deterministic to be reproducible and repeatable so that the same set of inputs produce the same output. The third characteristic is that the algorithm, when implemented on a computer, must consume a reasonable amount of time and storage based on the problem needs. For example, an algorithm can be finite and precise, but if it requires 100 years to generate a result, it is clearly useless. For instance, an algorithm that counts the number of even numbers is not finite because we have an infinite number of even numbers.

Assuming we have an algorithm, we then need to prepare it for execution on the computer. First, we must prepare the input before it is ready to be consumed by the computer, so we give the algorithm to a programmer whose job it is to read the algorithm, understand it, and then write a program in a known computer language such as C/C++ or Python. The program tells the computer what to do, but in a formal way rather than a freeform way as an algorithm. At that point, we move to another level of abstraction.

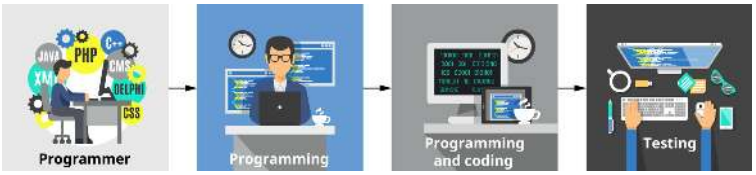


Figure 5.6 The resulting program is consumable by a computer during and after testing. (credit: “Programmer Flat Set” by Macrovector_official/FreePik, CC BY 2.0)

Application Programs Abstractions

Next, the programmer writes a program. The main difference between an algorithm and a program is that the former is written by an algorithm designer and the latter is written by a programmer so that the program can be executed by a machine. These programs are called application programs, or simply, programs, and there are billions of them in existence today. Once the programmer finishes writing the program, there are two more steps before it can be executed by a computer.

High-Level Programming Language

The program generated in the previous step is written in a programming language. There are many programming languages in the world currently. A **high-level language (HLL)** is the most evolved method by which a human can direct a computer on how to perform tasks and applications. The phrase *high-level* means that it is closer to a natural language rather than a machine level language such as strings of 1s and 0s. That is, HLL is more user-friendly, and it is made to make the life of the programmer easier regardless of the hardware or the machine. If you look at the code of a program, you find that it is still in English, yet a restricted version of English with very specific keywords and formats to remove the ambiguity that usually exists in natural human language.

Even though HLLs rely on restrictive versions of English, they still use English at a high level. The machine does not understand English and needs a low-level language; therefore, we need yet another next step: assembly language.

THINK IT THROUGH

One Hundred or One?

Since HLLs aim to make the life of the programmers easy, why do we have many HLLs? Why not one language that all programmers use?

Programming languages are typically designed to help create readable programs. However, some languages are designed with specific applications in mind. That is, some programming languages are easier to use for designing games, while other languages are meant to address mathematical problems or artificial intelligence. However, we can write any program in any language. But our task will be easier if we use the language that is designed with specific applications in mind.

Assembly Language

When you look at most mainstream programming languages, you find constructs such as functions, methods, subroutines, and objects. These concepts were invented to help human beings (the programmers) write their programs after understanding the algorithm. High-level languages make life easier for programmers. By using functions, objects, and other constructs, programmers can write programs faster and make them understandable and reusable for others. Computers, on the other hand, need specific instructions to perform tasks (for example, add this number to that other number and store the result in that place). Writing programs in this way is not easy for programmers because it is error prone, takes a lot of time to write correctly, and is not portable from one computer to the next. So, how do we deal with these conflicting requirements for programmers and computers? The answer is compilers.

A compiler is a piece of software that takes a program written in a given HLL and generates another program that does the same task as the initial program but is written in a language that is much friendlier to the computer called assembly language. Assembly language is then translated into machine language code so that the program can be executed (Figure 5.7).



Figure 5.7 A compiler takes code written in an HLL and converts it to "computer-friendly" and simple code called assembly language. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

One low-level language designed to be computer friendly rather than human friendly is called **assembly language**. It does not use all the constructs found in HLLs such as objects or sophisticated data structure, which leads to two challenges.

The first challenge is to manage the output of the compiler, which is an assembly language program that you can open and read. It is basically a text file, so it is written in English. Remember, computers do not understand English; they only relate to 1s and 0s. To deal with this challenge we use yet another program, shown in Figure 5.5, called an assembler (note that compilers may invoke assemblers directly). The **assembler** takes, as input, the assembly program generated by the compiler, and as output, a file that contains the equivalent of that assembly program in terms of 1s and 0s. This generated file is no longer in English, and you cannot open it with your favorite text editor. It is called a machine language file. This is the file that a computer understands. Figure 5.8 shows an example of a program written in an HLL (actually, a middle-level language, which is the C programming language), which is then translated to x86 assembly language and then to binary (refer to Chapter 4 Linguistic Realization of Algorithms: Low-Level Programming Languages).

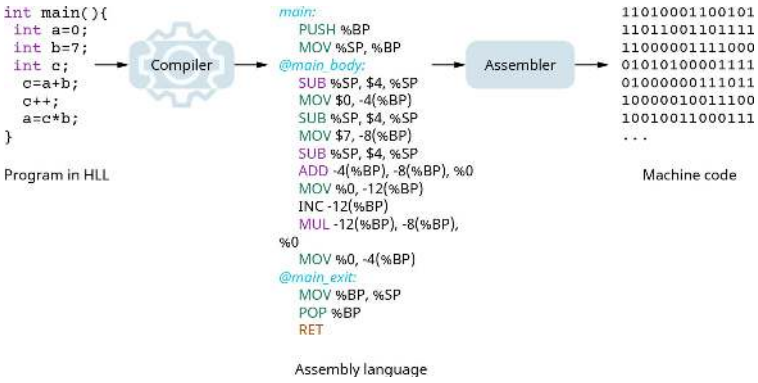


Figure 5.8 Programmers write in programs using HLLs but computers execute binary code, so we need to perform a translation. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

The second challenge relates to managing the assembly language itself. If you look at the assembly language program, you find that it consists of instructions such as add, divide, and jump. But are these instructions recognized by all the processors (i.e., the main part of the computer that executes the program) in the world? If you give these instructions to a processor such as Intel, AMD, ARM, Qualcomm, or IBM Power, will they all recognize these instructions? The answer depends on a new concept referred to as the instruction set architecture.

Instruction Set Architecture

The instruction set architecture (ISA) is the set of instructions recognized by each processor family. For example, both Intel and AMD processors use the same ISA, called x86-64, which is different from the ISA recognized by ARM or IBM. This makes us revisit the concept of compilers. A compiler takes as input a program written in an HLL, and we have a compiler for each HLL. The output of the compiler is an assembly program in a specific ISA, and we have a compiler for each different ISA (e.g., ISA 1 and ISA 2). So, if we have programs written in three HLLs and we need to generate assembly for processors of two different families,

then we need six compilers as shown in Table 5.1.

Compiler#	Input to the Compiler	Output from the Compiler
1	HLL 1	ISA 1
2	HLL 2	ISA 2
3	HLL 3	ISA 1
4	HLL 1	ISA 2
5	HLL 2	ISA 1
6	HLL 3	ISA 2

Table 5.1 Compilers and Their I/O

As we saw earlier, the output of the compiler is the input to the assembler so we need assemblers for each ISA in existence to generate a machine language file (e.g., .exe file) that the processor can execute.

Processor Abstractions

As we cross the layer of ISA in Figure 5.5, we cross the boundary between software and hardware. Before we discuss hardware, we need to understand two words: translator and interpreter. Both words mean "translating from language 1 to language 2" regardless of what those languages are. The main difference is the process by which translation is done. A translator takes a whole program in language 1 and generates another program in language 2. For example, the compiler takes an HLL as input (language 1) and generates the corresponding assembly language program (language 2). The interpreter takes one line (or command) in the program in language 1 and generates one (or more) instructions in language 2. Python is a popular example of an interpreted language.

Understanding the hardware level allows us to see how computers execute programs. The main part of the computer hardware that does the execution is called the processor. The processor takes one instruction from the machine language file, executes it, and writes the results back in a designated place. Then, it fetches the following instruction and does the same. It keeps doing this until the program ends or an error occurs. This is an oversimplification, but it conveys the general idea. As you can see, it takes one instruction at a time, which is why the vertical arrow coming out of the machine language box in Figure 5.5 shows the word *interpreter*. But how does the processor do its job of fetching an instruction and executing it? To answer this question, we need to look at the main components of a processor.

LINK TO LEARNING

The transistor is the building block of the hardware of any computer. A transistor is merely an on/off switch; when it is on, it lets the electrical current pass. When it is off, it blocks the electrical current. This is very similar to the light switch you find in your room. But with these transistors, we can do more: with transistors, we build computers. Read this article to learn more about transistors (https://openstax.org/r/76Transistors) and how they work.

Microarchitecture

The architecture (i.e., design) of the microprocessor (i.e., the processor) is called **microarchitecture** (Figure 5.9). Its main job is to design the different components of the processor and decide how to connect them so that the processor can do its job. For example, one important piece inside the processor is the part that fetches the next instruction of the program. Another crucial part, called the control unit, is responsible for decoding, or understanding, what this instruction wants to do and then tells the other components of the processor to execute this instruction. So, the control unit's job is to take an instruction as input and generate signals to control the rest of the processor to make it execute the instructions. Other parts of the processor include the execution units that do the actual computations such as divide, multiply, add, and subtract.

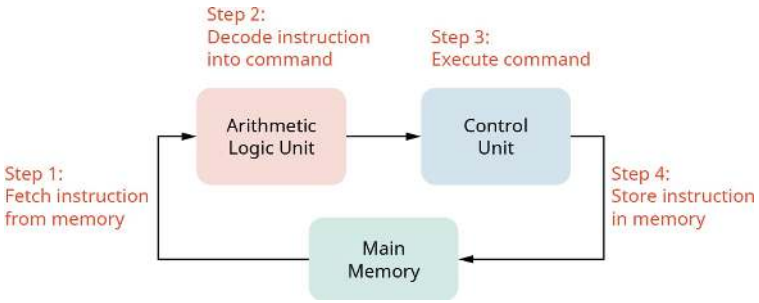


Figure 5.9 Although it may just take seconds to execute, there are several steps included in a microarchitecture process to deliver results for the user. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

To summarize, inside the processor you find those different pieces that fetch instructions, decode them, and execute them. But what is inside each one of these black boxes?

Digital Logic Abstraction

The main building block that forms the processor is called a **logic gate**. There are very few logic gate types: AND, OR, NOT, NAND, NOR, XOR, and XNOR. Using these gates, and most of the time only a subset of them, you can design all the pieces that form the processor discussed earlier. But what is inside these gates? How are they built?

The Lowest Level of Abstraction

The main building block of all logic gates, and hence of all processors, is the transistor. This is shown at the bottom of Figure 5.5 as device level. Though you may have heard the word *transistor* before, you may not know exactly what it does. Simply speaking, a transistor is an on and off switch. This is very similar to the light switch in your house that can turn a light on or off. A **transistor** lets an electrical current pass, the ON state, or can block the electrical current, the OFF state. Transistors are turned ON/OFF based on the voltage input to the transistor. If the voltage is higher than a threshold, the transistor is in the ON state. Otherwise, it is in the OFF state. There are only two states: ON/OFF, which correspond to 1 and 0. This is why computers understand only 1s and 0s. By interconnecting several transistors in some way, we build an AND gate. If we connect them in a different way, we build an OR gate, and so on.

If we try to see how transistors are built and work, then we move to the semiconductor level. At this level we use a special material such as silicon and a special, and very expensive, process to turn it into a working transistor. A single processor contains billions of transistors. How can a material like silicon make a transistor? This takes us to the level of atoms and quantum physics.

The Role of the Operating System

Now that we have rundown the problem definition to quantum physics, you may wonder where the operating system (OS) (i.e., OS X, Windows, Linux) fits in this bigger scheme. The OS is similar to any application program in the sense that it has to be written in an HLL, typically C/C++, and passed through the compiler and assembler to generate machine code. However, the OS differs from traditional application programs in that it

has more privileges in the computer system.

The **operating system (OS)**, shown in Figure 5.10 is the only piece of software that can directly access the hardware. Any other program that needs to access the hardware, such as printing something, must talk to the OS, and the OS achieves the requested task on behalf of the program. The reason all computers are designed this way is to increase security (only one program deals with the hardware so other programs cannot affect the machine) and reliability (a program cannot affect a piece of hardware, which would then affect other programs). In order for the OS to do its job efficiently, it stores the data and programs on disk in an organized way using a file system. The file system helps organize files so that it is easier to find them when needed.

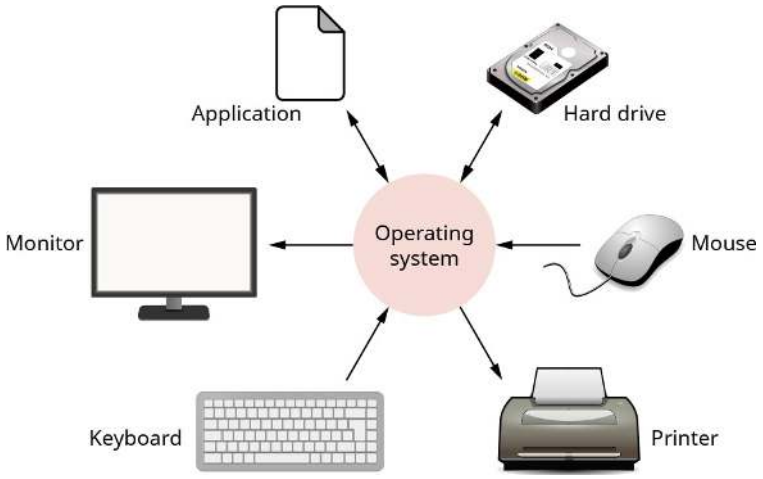


Figure 5.10 The operating system functions as a manager that connects the hardware in the computer to the software. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

A file is a generic name for any entity we want to store in the computer. For example, any program that you use consists of one or more files. Each song you listen to is a file. Any image or video you watch is a file, and so on. The process in which the OS manages which program to use with what part of the hardware at any given time is called **scheduling**. That is, the OS decides when your web browser must use the processor and when your media player uses the screen or speaker. There is a generic name for all programs running on your computer: process. So, if you are listening to music while browsing the web then you are using two processes: your media player and your browser. Therefore, part of the job of the OS is process scheduling.

The technique that the OS uses to isolate different programs from each other so that they do not overwrite each other's data or corrupt each other's files is called **virtual memory**. Additionally, the OS can leverage several computers referred to as virtual machines together to act as one big computer and to serve several uses at the same time. Users may think they have control of the whole machine, even though the reality is not so. This is why this technology used by the OS is called virtual machine.

There are many OSs in the world but the most famous are Windows from Microsoft, macOS from Apple, and Linux.

New Disruptive Computer System Abstractions

Almost all computers in the world are designed in the way we have learned so far and involve very similar levels of abstraction. However, there are very futuristic designs that scientists are tinkering with today that differ from the traditional transistors. Scientists are trying, for example, to build computers using DNA. We have DNA computing, and we have a prototype for DNA storage, too.

We have several prototypes from various companies of quantum computing where instead of using bits, 1 and 0, the machines use quantum bits (qubits) that take a value between 1 and 0. We must not forget that traditional computers in general operate in binary state (i.e., using 0 and 1). To use these computers, we need to build different types of compilers, operating systems, programming languages, and so on. Another form of

a non-traditional computer is a **neuromorphic computer**, which is built to act like a simplified version of the brain. So, it consists of hardware neurons connected together. These computers are not programmed but trained. What will computers look like 100 years from now? We do not know, yet.

5.3 Machine-Level Information Representation

Learning Objectives

By the end of this section, you will be able to:

- Interpret binary numbers
- Explain the use of standard character codes to represent letters and symbols
- Define fractional binary numbers and explain how they are used

In this section we look at two very important and widely used types of information: numbers and text. The reason we concentrate on these two types is because they are standardized, and almost all computers store them in the same format, unlike other types of information that have many different formats, some of which are proprietary.

It is important to know that a series of bits does not have a meaning by itself. For example, in order to interpret or translate the binary number 0011 as a decimal number, you need to know whether it is an unsigned or a signed integer. Therefore, we need to understand how the different types of information are presented in binary.

Integer Numbers Representation

As human beings, we always think in numbers, specifically decimals. Whenever you mention a number, it is usually in decimal form; that is, base-10. But what does base-10 mean? It means the value of the number is calculated by scanning the number from right to left. And as you pass by a digit, multiply it by 10 to the power of the position of that digit. The number at the far right, called the least significant digit, has position 0. The one after is at position 1, and so on. Figure 5.11(a) shows this process.

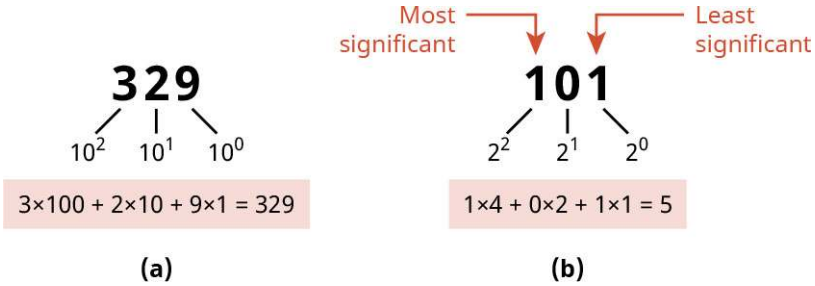


Figure 5.11 The only difference between (a) decimal and (b) binary is the base. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

What about binary numbers? Binary numbers use base-2 and are presented as a series of 1s and 0s. In order to go from binary to the equivalent decimal, we need to differentiate between the ways unsigned integers and signed integers are represented in binary.

LINK TO LEARNING

The best way to get a clear view of how numbers are translated to binary so that computers can deal with them is to see the process in action. Visit this site to convert any number from decimal to binary and vice versa (https://openstax.org/r/76DecimalBinary) and to read more about how to do it manually.

Unsigned Integer Numbers Representation

An **unsigned integer** is a non-negative integer that starts from 0. If you see a number in binary and you are told that this number represents an unsigned number, you can get its decimal equivalent in the same way as base-10 integers but using base-2 instead. Figure 5.11(b) shows the operation of getting the decimal equivalent of the binary number 101 by starting from the right and moving left. The least significant bit has a position 0, the next one has position 1, and so on. As you pass by each element, add $X \times 2^p$, where p is the position of the digit and X is the digit itself (0 or 1), to the total sum.

One important thing to keep in mind is the range of numbers that can be presented by an unsigned number. One bit can present two values only: 1 and 0. Two bits can present four values: 00, 01, 10, and 11 which correspond to 0, 1, 2, and 3. Three bits can present eight values. In general n bits can present 2^n values, which is the range from 0 to $2^n - 1$. As you can see, there are no negative numbers. To be able to present negative numbers, we need to use signed numbers.

Signed Integer Numbers Representation

A **signed integer** is an integer that can be negative or positive. The word *signed* means that they have a sign of + or –. To present negative numbers, designers experimented with several options before picking the de facto choice. The obvious option is to use the most significant bit (i.e., the leftmost bit) as a sign bit where 0 means positive and 1 means negative. The rest of the number is treated with a method called sign-magnitude. Is it a good option? Let us look at the first column of Figure 5.12. It represents the sign-magnitude for a 3-bit number. We see two things.

Sign-Magnitude:	One's Complement	Two's Complement
000 = +0	000 = +0	000 = +0
001 = +1	001 = +1	001 = +1
010 = +2	010 = +2	010 = +2
011 = +3	011 = +3	011 = +3
100 = –0	100 = –3	100 = –4
101 = –1	101 = –2	101 = –3
110 = –2	110 = –1	110 = –2
111 = –3	111 = –0	111 = –1

Figure 5.12 Two's complement is the representation of choice for signed numbers. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

First, there are two presentations of 0 (+0 and –0), which is a waste of data. Second, the numbers are not mirrored around 0. That is, usually, the number after 0 is +1 and the one before 0 is –1. This is not realized here. The importance of this mirror is to make the hardware a bit simpler.

The second method to try is a bit less intuitive. It is called the **one's complement** method of a binary number, which is obtained by flipping each 1 in the original number to 0 and each 0 to 1. For example, 101 has its one's complement as 010. The one's complement method is explained in more detail:

- Check the most significant bit and if it is 1, the number is negative and is represented in its one's complement form. To get the original number, you need to get its one's complement.
- The one's complement of the one's complement is the original number. For example, the one's complement of 101 is 010. The one's complement of 010 is 101.
- Looking at the second column of Figure 5.12, let us take a number such as 110. Its most significant bit is 1 so the number is negative and in its one's complement form. To know its original value, we need to get its one's complement again. The one's complement of 110 is 001 which corresponds to decimal number 1, so the result is –1.
- If the most significant bit is 0, then the number is positive, and to get its decimal equivalent, we treat it as if it is unsigned. With the number 010, the most significant bit is 0, so the number is positive and the decimal equivalent of 010 is 2. Therefore, the result is +2.

Is the one's complement a good method? There is a mirroring effect around the +3/–3. But we still have the two presentations of 0. The third method is called the two's complement, which is the least intuitive method. It is important to note that what is good for machines is not intuitive or easy for human beings!

The **two's complement** of a binary number is simply the one's complement with 1 added to the result. For example, to get the one's complement of 011, we do it in two steps. First, we get the one's complement: 011 is 100. Second, we add 1 to the result: 100 + 1 = 101, and 101 is the two's complement of 011. Now that we know the definition of two's complement, let us see how we can get the decimal equivalent of a binary number. We perform a very similar technique as the one's complement but use the two's complement.

Look at the most significant bit. If it is 0, the number is positive, and the decimal equivalent can be calculated as if the number is unsigned. For example, 011 is positive and the decimal equivalent is +3. If the most significant bit is 1, then the number is negative, and it is written in its two's complement form. Also, the two's complement of the two's complement brings the original number so 101 has the most significant bit of 1. The number is negative and written in its two's complement form.

The two's complement of 101 is 011, which corresponds to 3. The number 101 represents –3, as you can see in the third column of Figure 5.12. A close look at the figure shows that even though the two's complement is not the most intuitive method, it is the most efficient. Since we have only one presentation of 0, we can see from the figure that with three bits, the two's complement method can present up to –4, which we did not find in the other two methods.

Another important issue with the two's complement is that the hardware implementation of it is very efficient because we can use the same piece of hardware for both addition and subtraction and for both signed and unsigned integers. Therefore, a two's complement presentation of a signed number is the standard presentation in all computer systems.

With n bits, the range of numbers that can be presented is $[-2^{n-1}, +2^{n-1} - 1]$. You can see that this range contains 2^n different numbers, exactly like n-bit unsigned integer numbers. The only difference is that in the signed range, half the range is negative, while in an unsigned integer, the whole range is positive.

Lastly, let us look at how additions and subtractions are done. The process is similar to a decimal by starting from the right and a possible carry propagates to the left. Additionally, the subtraction is nothing but an addition; that is, A – B is the same as A + two's complement of B. When we add two numbers in decimal, it is straightforward. With binary, it is also straightforward. Just remember the information in Table 5.2; op1 and op2 are the two inputs to the addition operation.

Op1	Op2	Carry	Sum
0	0	**0**	**0**
0	1	**0**	**1**
1	0	**0**	**1**
1	1	**1**	**1**

Table 5.2 Op1 and Op2 Inputs to the Addition Operation

Let us apply this to numbers longer than one bit. In Figure 5.13, the left side of the example is a traditional decimal addition as done on paper by hand. The right side of the example shows what computers do. Computers, as we already know by now, use only 1s and 0s. So, if they add 1 to 1, for example, the result is not

2 because computers do not know 2. The result of 1 + 1 is 10, which is the binary representation of 2. From this 10, the 0 is the result and 1 is used as carry.

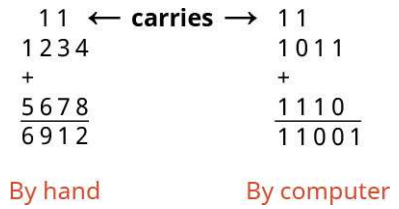


Figure 5.13 This illustrates the difference between traditional addition versus computer computation. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

THINK IT THROUGH

Why Integers?

A real number, represented on a computer as a single precision floating point, takes four bytes. An integer also takes four bytes. Yet, the range of numbers that a floating point can represent is much larger than the integer.

- Why do we use an integer in the first place?

Computers are much slower dealing with floating points because the hardware for a floating point is much more complicated than the one for an integer, and it is better to make things faster by performing computations with an integer. We also only use floating points if it is an absolute need.

Character Representation

Remember that what is stored inside the computer is not only integers. If you look at a keyboard you see a bunch of characters, digits, and symbols and when you press a key, that symbol appears on the screen. But how do computers recognize these characters and deal with them?

Each printable and non-printable character on the keyboard has a unit code or unique binary number. These codes are known as **American Standard Code for Information Interchange (ASCII)**. Capital letters have different codes from lowercase letters. For example, “A” has a different code than “a.” Decimal digits also have their own code. The code is 7-bit length, which covers 128 characters. This encoding has been extended to 8 bits to encompass non-printable characters as well (i.e., characters on the keyboard that cannot be printed on the screen: can you guess them?). The reason to have standardized codes across all machines is to allow computers of different brands, specifications, and sizes to work together. This code was approved in 1963, before personal computers existed, and then revised in 1965, 1967, 1968, 1977, and 1986. Now, all computers use this code to represent the characters.

You may realize that 128 characters, or even 256 ones, cannot include all written languages which is why there are new encoding standards such as Universal Coded Character Set (UCS) and Unicode that encompass all written languages and incorporate ASCII as the first 128 codes, which is backward compatible, allowing for the integration with older legacy systems.

Real Numbers (Floating Points)

There is no computer worth its salt that cannot present and manipulate real numbers. A real number, such as 3.14, is also called a **floating point number** because there is a decimal point somewhere in the middle. In most HLLs, a single precision floating point number requires four bytes of storage such as integers (signed and unsigned).

Let us start with an easy question: if given a binary floating-point number, for example 1010.010101, how do

we get the decimal equivalent? Figure 5.14 illustrates how fractional binary numbers are represented. We use the same technique as getting the decimal equivalent of an unsigned integer. The only difference is that the digits at the right, after the floating point, have negative ranks starting from –1. If we have a number such as 11.111, the rank of the first digit after the floating point is –1, the following one is –2, and the leftmost one is –3. The digits at the left of the floating point have the usual rank that starts from 0. Therefore, the decimal equivalent of $11.111 = 2^1 + 2^0 + 2^{-1} + 2^{-2} + 2^{-3} = 2 + 1 + 0.5 + 0.25 + 0.125 = 3.875$.

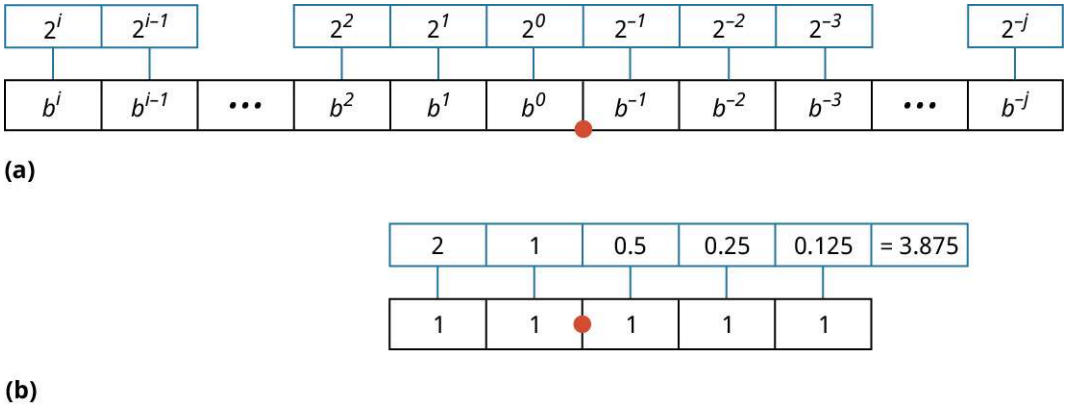


Figure 5.14 Floating point uses the same base 2, but after the decimal point the exponent is negative. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

LINK TO LEARNING

The best way to get a clear view of how numbers are translated to binary so that computers can deal with them is to see the process in action. Visit this site to convert any number from floating point to binary and vice versa (https://openstax.org/r/76FloatPoBinary) and to read more about how to do it manually.

But how do we store numbers like 11.111 inside the computer memory or register? The hard part is the position of the floating point itself because it is not known beforehand and can change during computations, which makes it very hard to decide how many bits to reserve for the digits at the right of the floating point and the digits at the left of the floating points. If we try to make it a fixed number, say 16 bits and 16 bits, we do not get good precision. Going back to the mathematical representation of decimal floating point numbers, you may recall that we can move the floating point to the left or to the right and keep the final value unchanged by multiplying by 10 to some power. For example, 1.875 is the same as 18.75×10^{-1}, which is the same as 1875×10^{-3}, which is the same as 0.1875×10, and so on. In binary, we can do the same by multiplying by 2 to the power of something. The number 11.111 is the same as 111.11×2^{-1} and so on. In fact, we can express any floating point binary number in the form $1.xxxx \times 2^y$. Except for special cases, such as 0, expressing any binary number in this format requires storing three pieces of information:

- The sign bit specifying if the whole number is positive (sign bit of 0) or negative (sign bit of 1)
- The exponent (the y in $1.xxxx \times 2^y$)
- The fraction (the xxxx in $1.xxxx \times 2^y$)

Note we don't need to store the "1." because we know that it exists except for some special cases. And this is why one of the names of the floating point format is "the hidden 1 technique." If we are talking about single precision floating point, it takes 4 bytes (32 bits) and the format is shown in Figure 5.15. One bit is needed for the sign, 8 bits for the exponent, and the rest (32 bits) for the fraction. This format is called **IEEE 754** format, developed by the Institute of Electrical and Electronics Engineers (IEEE). It is the standard format used by almost all computers that support floating points with very few exceptions.

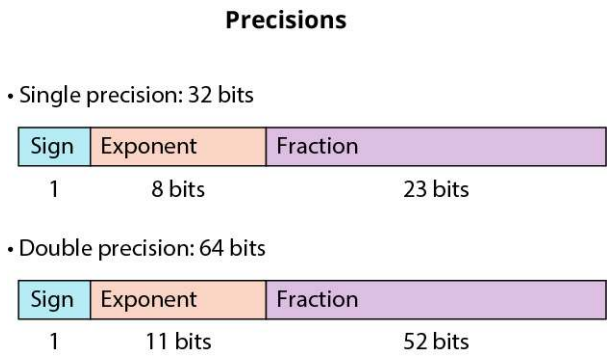


Figure 5.15 IEEE 754 is the standard of choice in most machines to present floating points. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

There is one piece of complexity regarding the exponent—the sign bit is responsible for the sign of the whole number. But, what about the exponent? We need to be able to have positive and negative exponents. If we use the two's complement format for the exponent, the whole floating point format is overly complicated. The hardware that deals with floating point operations is very complicated and much slower than the one dealing with integers, and we do not want to make it even more complicated. The solution is to shift the range of the numbers presented by the 8 bits of the exponent. What does that mean? With 8 bits, we can present numbers from 0 to 256. What if we subtract 127 from each number? That is, you read the 8 bits, get the decimal equivalent (same way as an unsigned integer), then subtract 127. The result will be the range that can be presented is now from –127 to +128. You get the idea.

Another name for the IEEE 754 format is "excess 127." Note that 127 is for the single precision. For double precision that takes 64 bits, we subtract 1023, which is roughly half the range. The double precision is shown at the bottom of Figure 5.15. Let us see an example. What is the decimal equivalent of 1011101101011000...0?

- First, let us divide it into its three main components: sign, exponent, and fraction. This makes it: **1 01110110 10110000...0**.
- Sign bit is 1, so the whole number is negative.
- Exponent = 01110110. Its decimal equivalent is 118. We subtract 127. This makes the exponent: 118 – 127 = –9.
- The rest is the fraction, but we need to add the hidden one, so 1011000...0 becomes 1.1011, which has a decimal equivalent of $1 + 2^{-1} + 2^{-3} + 2^{-4} = 1.6875$.
- This makes the whole number = -1.6875×2^{-9}.

Before we finish our discussion about floating points, there is the question of 0. How to present the 0? Even if we make all bits 0, the hidden 1 makes the final value a non-zero one. How can we deal with this problem? The representation approach that we have learned so far is called the normalized encoding of the IEEE 754 format. This is used if the exponent is non-zero and is not 11111111. If the exponent is 0 (i.e., 00000000) we are in denormalized encoding (also called subnormal). When we are in this special case, there are some differences in the translation to decimal:

- The exponent is 1-bias instead of 0-bias. The bias is 127 in single precision and 1023 in double precision.
- There is no hidden 1, so the fraction part is 0.xxxx (the 23 bits in the fraction in single precision) instead of 1.xxxx....

With these exceptions, we cannot present the 0 (but setting all 32 bits to 0) but can present very small numbers.

The case where the exponent is all 1s is called "special values encoding." If the exponent is all 1s and the fraction is all 0s, it represents infinity. If the exponent is all 1s and the fraction is non-zero, this is called NaN (Not a Number) and raises an exception. This happens when there is a bug in your program that does a division by 0 or the square root of –1, for example.

5.4 Machine-Level Program Representation

Learning Objectives

By the end of this section, you will be able to:

- Discuss x86-64 Intel processors and their architectures
- Differentiate between assembly and machine languages
- Explain basic concepts of assembly language and the types of operations

When you write a program in an HLL, there are several steps that need to be performed before the processor can start executing the code. Refer to Figure 4.18 for a high-level view of the process.

Let us assume you write your program in C and your program is spread over several source files for ease of management. As you now know, the first step is to go through the compiler. The compiler is totally oblivious to the fact that the multiple source code files belong to the same program; it just takes each file separately and generates the corresponding assembly language file for each one of them. If the input is three C files, the output of the compiler will be three assembly language files.

The next step is to take these language files and translate them to machine code files, also known as object files or binary files. Here too, the assembler is oblivious to the fact that the input assembly files do not belong to the same program, so it translates them separately. The first tool in this workflow that recognizes that all the files belong to the same program is the linker. The linker takes all the generated object files, looks for needed libraries, and links everything together into one executable file. Linked libraries are needed because it is very unlikely that programmers write self-contained code. You still use I/O, for example, for printing something on the screen, but you have not implemented those functions yourself—or you use mathematical functions someone else implemented. A library linked at this step is called a static library.

At this stage, you have an executable file residing on your disk until you decide to execute it by typing a command, clicking an icon, or even saying a command. At that moment, a part of the operating system, called the **loader**, loads the executable into the memory and arranges its content in a specific way to make it ready for execution by the processor. A **dynamic library** is when more libraries may be linked during execution or while the program is running.

This section takes a closer look at assembly language. As an example of a widely used assembly language ISA, we will look at x86 ISA used by Intel and AMD processors. But before we delve into this, we need to ask a simple question: Why learn assembly language? Will you ever need to write code in assembly language? Most likely not, except in rare cases where you are developing some part of an operating system, a device driver, or any other application that requires very low-level manipulation. However, by looking at the assembly language generated by the compiler for your code, you can find innovative ways to optimize your code, detect hidden bugs, and reason about the performance of your code once you execute it.

Intel Processors and Related Architectures

You may recall that each processor family understands a set of instructions, which is the ISA. The family of processors from Intel and AMD share the same ISA called x86 (x86-64 for the relatively newer version for later processors). This ISA has a long history that dates back to the 1970s. Figure 5.16 gives a quick glimpse at how things evolved. The figure does not show every single processor from Intel but instead focuses on some milestones.

Name	Transistors	MHz
• **8086** (1978) • First 16-bit processor. Basis for IBM PC & DOS • 1MB address space	29K	5–10
• **386** (1985) • First 16-bit processor, referred to as IA32 • Capable of running Unix	275K	16–33
• **Pentium 4** (1978) • First 64-bit processor, referred to as x86-64	125M	2800–3800
• **Core i7** (1978)	731M	2667–3333
• **Xeon E7** (1978)	2.2B	~2400
• **Core i9** (1978)	2.95B	~3900

Figure 5.16 x86 is a CISC ISA with backward compatibility dating back to the 1970s. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

First, as technology evolves and the need for faster processing power arises, we move from 16, to 32, to 64-bit processors. This number relates to the size of the registers (fast storage entities inside the processor), the width of the buses (parallel wires connecting the processor to memory), and the amount of memory the processor can access (for n-bit machines, the processor can access 2^n bytes of memory). The ISA also evolves in parallel to incorporate the larger registers (hence the move from IA32 to x86-64) and the computations with larger numbers. Second, we can see the tremendous increase in transistors in each generation. Having more transistors means more features implemented inside the processor and hence higher performance and potentially richer ISA.

The complex architecture structure that assists in executing operations such as mathematical computations and memory storage is called **complex instruction set computer (CISC)**. This is done by combining many simple instructions into a single complex one. This concept came from something called the semantic gap, which is the difference between the HLL program and its assembly equivalent. It is good for programmers to understand assembly language as this skill will help you code in any language. However, assembly programming is so much different from HLL programming that most programmers have difficulty understanding it. The wider the difference, the wider the semantic gap. To reduce this gap and make assembly language more accessible to programmers, x86 was designed to make its instructions a bit more complicated because statements in HLL are complicated. Complicated means a single assembly instruction can do several things. For example, an instruction like *addw %rax, (%rbx)* means access the memory at a specific address, get the data stored there, add that data to a number, and store that number in a specific place. So, it is accessing the memory, making an addition, and storing the result somewhere. Because the instructions are complicated, this set of these instructions is called CISC.

Complex instructions such as the ones corresponding to a for-loop in HLL were the norm until the 1980s when another viewpoint came into existence that said that complex instructions make the processor slow. Moving into the 1990s, and the appearance of portable devices with their sensitivity to power consumption and battery life, another disadvantage of CISC arose: complex instructions make the processor not only slow, but also power hungry. And, thus, the other viewpoint of simpler instructions called **reduced instruction set computer (RISC)** came to be the norm. Right now, all the processor families in the world are RISC except x86.

LINK TO LEARNING

There has been a debate among companies who are designing hardware as to whether CISC or RISC is better. Read this article chronicling this debate (https://openstax.org/r/76DebCISCvsRISC) from MicrocontrollerTips.

Assembly and Machine Code

In our discussion of Figure 4.18 we saw assembly (output of the compiler) and **object code**, binary code, and machine code, which all designate output of the assembler. Machine code is the binary presentation of the assembly code. In some cases, there are assembly instructions that do not have a counterpart in the machine code called **pseudo-assembly**. For example, there are instructions in assembly that execute a *go to* if a number is less than another number. The only conditions known in machine code are equal and not equal, but not less than, less or equal, and so on. We can see this in an instruction set like MIPS.

The assembler's job is to ensure that the machine code file only contains instructions that are native to the processor; that is, part of the ISA. So, we can think of the machine code as being a subset of the assembly code. You never find an instruction in the machine code file that is not part of the ISA of the processor for which you want to generate the binary. The reason there are pseudo-assembly instructions is to give the compiler a bit more freedom to generate efficient code. Let us assume that you write a program in C and you think about functions calling each other. If you write a program in C++ or Java, you think in terms of objects, methods, inheritance, and so on (refer to Chapter 4 Linguistic Realization of Algorithms: Low-Level Programming Languages). We call this the programmer view of the language. What if you write (or read) assembly code? What do you see? This view is summarized in Figure 5.17.

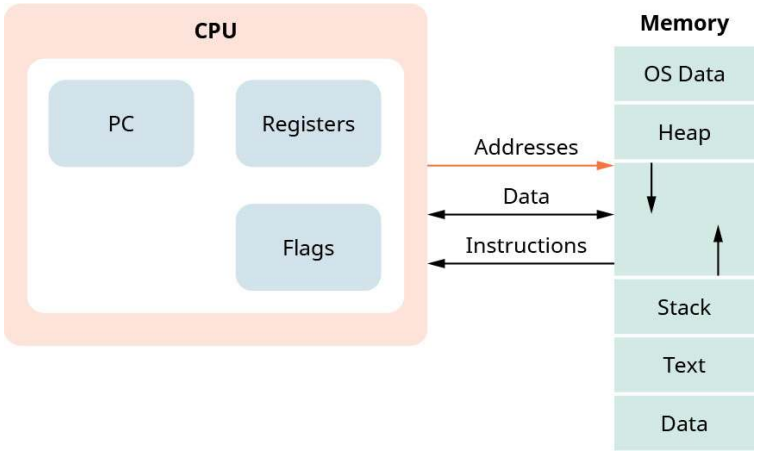


Figure 5.17 The assembly programmer sees a simpler, but more realistic view of the machine than the HLL programmer. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Figure 5.17 shows the following:

- There is a processor (CPU) and memory.
- The CPU and the memory are connected by a **bus**, which is a data pathway. When you access the memory, you may want to get data and must provide an address. Or you may want to write data to memory, so you must provide both the data and the address to which this data will be written. In both cases, the CPU must provide an address, therefore, the address bus is single directional. But the data bus is bidirectional because you can send data to memory or get data from memory.
- Data is not the only thing you need to bring from the memory to the CPU. The main job of the CPU is to execute instructions on data. For example, adding two numbers involves data (the two numbers) and the command for addition (instruction), which is why there is a single directional bus from memory to CPU for getting instructions from the memory.
- The memory holds several things: data, the instructions of the programs (shown as "Text" in Figure 5.17), some data needed by the OS to manage your program, and the resources it needs. The stack and heap are places in the memory to store data depending on the program at hand. The **stack** is used to store local variables (and some other stuff that we will discuss later), the **heap** stores dynamically allocated data, an Data in Figure 5.17 is another area in the memory to store global variables.
- Inside the CPU, there are registers which carry hardware parts that store data, instructions, addresses,

and so on. Each register stores one item. An x86 programmer has access to 16 registers, as we will see shortly. Because the CPU is executing a program, which is a series of instructions, the CPU must keep track of what the next instruction to be executed is.

- Keeping track is the job of a specific register, shown separately from the other registers, called the **program counter (PC)**. The PC is updated after executing each instruction to point to the next instruction to be executed. Also, it is useful to keep some information about the result of the instructions executed, such as whether the result generated by the current instruction is positive, negative, or zero, which is the job of the flags. A **flag** tells a program if a condition has been met.

Registers

A **register** is a memory unit that functions at very high speed. Figure 5.17 shows registers as one black box. If we open this box and see what is inside, we see 16 registers. Any instruction in x86 assembly uses only those 16 registers. The name of each register is shown in Figure 5.18 and starts with an "r." The naming convention of registers is a bit odd, but it is due to some historical naming (for the registers on the left). To keep backward compatibility, old register names cannot be changed.

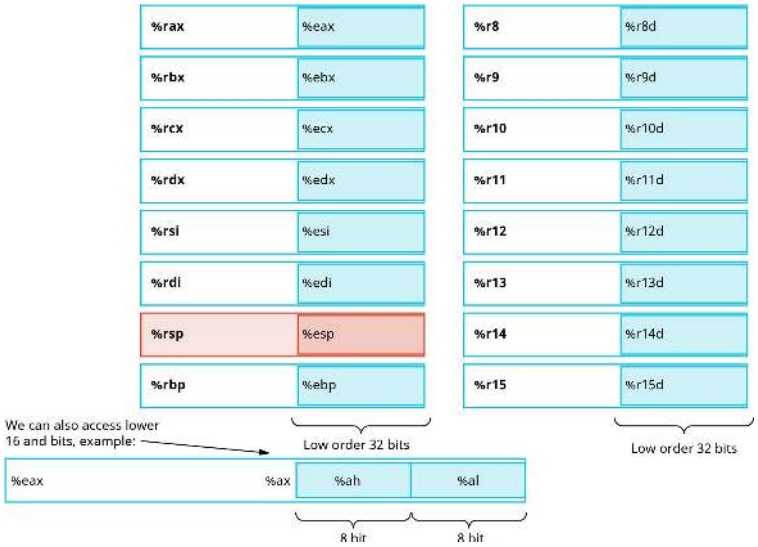


Figure 5.18 Registers in x86 are a bit complicated due to the need to keep backward compatibility and the fact that x86 is CISC. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Each one of these registers can hold 64 bits. When we had 32-bit machines, only eight registers existed, as shown on the left of Figure 5.18. In the 32-bit era, each register could hold 32-bits only and its name started with "e" (for extended). This is why we see that the lower 32 bits of the current registers hold the old names to keep backward compatibility. Not only that, in the 16-bit era, each register held 16 bits only. Those on the left had names: ax, bx, cx, and so on, which are the names of the lowest 16 bits of the current registers. Registers on the right side of the figure did not exist before the 64-bit era. If we go to the 8-bit era, we can even access the lowest 8-bits of the register. The register rax is shown as an example where parts of the register can be accessed using the naming convention: rax (16 bits), eax (32 bits), ax (16 bits), and ah and al (8 bits each). In the 8-bit era there were only four registers, which are the top four in the left column of the figure.

CONCEPTS IN PRACTICE

HLLs and Assembly

Most programmers use HLLs to write their programs. Why do you think world class programmers are very well versed in assembly? Professional programmers write in HLL but they often like to look at the assembly version of their code as well. This allows them to discover mistakes in their HLL code and also to find out

ways to enhance their HLL code.

However, there are some programs, or parts of programs, that need to be written in assembly and not in HLL for the sake of performance (assembly code written by a human in these cases is much faster than HLL translated to assembly by a compiler) and more control over the hardware. Assembly language common uses today include device drivers, low-level embedded systems, and real-time systems.

Operands

An assembly language program consists of a group of instructions. Each instruction does an operation and for this operation to be executed, it needs operands. An **operand** is a value used as input for an operator. Perhaps we want to add two numbers in an operation. To be executed it needs two numbers, which are the operands. For example: *add %RAX, %RBX* means add the content of the register RAX to the content of the register RBX and put the result in register RAX. Operands in assembly can be one of three things:

- A register
- An immediate operand (e.g., add 7 to rax; the operand here is mentioned explicitly in the instruction)
- Data from memory

Memory Addressing Modes

To get data from the memory, we need to specify the location, called the **addressing mode**, in the memory that contains the required data. Why are there several "modes" instead of just specifying an address directly? Well, the answer is related to the HLL.

In HLL programs, we use complicated data structures, such as arrays with one or more dimensions, structures, or linked lists. The compiler needs to translate this data structure to a much simpler assembly language. In assembly there are no complex data structures; there are only data items of 1, 2, 4, and 8 bytes. Then how can we map these complex data structures to the simple single dimension data items? One crucial way is to have rich addressing modes. In its most general form, an address in x86 is specified as D (Rb, Ri, S) where:

- D is a non-negative (but can be 0) integer whose range is from 0 to $2^{32} - 1$.
- Rb and Ri are registers, and they can be any one of the 16 registers.
- S is a scale that takes one of the following values: 1, 2, 4, or 8.

The address is calculated as: D + Rb + (S × Ri). While this is a general form, it can have more reduced forms such as (Rb, Ri). Here D's default is 0 and S's default is 1; D (Rb, Ri); or (Rb, Ri, S).

Now that we know about operands, let's look at the operations themselves that are implemented by the different assembly instructions.

LINK TO LEARNING

It is always good to see the concepts we learn in action. Visit this site to write an HLL program and see the corresponding assembly (https://openstax.org/r/76HLLProgram) at the same time. Any change you make to the HLL code will have an effect on the assembly.

Assembly Operations

In any ISA, all assembly instructions fall into one of three categories:

1. Data movement: from register to register, from memory to register, and from register to memory
2. Arithmetic and logic operations including addition, subtraction, AND, OR, and so on

3. Control instructions, which are the instructions that implement the “go to” operations, whether conditional or non-conditional (category also includes procedure calls)

Data Movement Operations

The data movement in assembly takes the form movx source or destination where:

- “x” specifies the number of bytes to be moved from source to destination. It can take one of the following values: “b” means 1 byte, “w” (word) means 2 bytes, “l” (long) means 4 bytes, and “q” (quad word) means 8 bytes. If you think about it, these are the sizes of all the data types we have in any HLL.
- the source and destination can be any of the operand types we mentioned earlier. There are only three combinations that are not allowed. The first is to move immediate to immediate as it does not make sense. The second is to move from memory to memory because the CPU must be involved. The last prohibited combination is when the destination is immediate as it also does not make any sense. Some examples:
  - “movq %rax, %rbx”
    Move 8 bytes from register rax and put them in register rbx, which is not really a move; it is a copy.
  - “movq (%rax, %rbx, 4), %rcx”
    This is a bit complicated. It involves three steps: first, calculate [rax + (4 × rbx)]; second, use this calculated value as an address and go to the memory at that address; and third, get 8 bytes, starting from the address you calculated (do not forget the “q” at the end of the mov instruction) and put them in register rcx. Now do you see why x86 is CISC where C means complex?

Arithmetic and Logic Operations

The arithmetic and logic operations involve very well-known operations such as add, subtract, multiply, and or, xor, shift left, and shift right. As you may have guessed, these are operations that require two operands. Look at the type of operands that we investigated earlier. For example:

```
addq %rax, %rbx
means rax = rax + rbx.
```

Additionally, there are some one operand operations such as increment and decrement. There are some complexities involved in multiplication and division where there are different instructions for signed and unsigned integers. And, for the division, yet another complexity is where to store the remainder.

Comparison and Test Operations

To be able to implement the complex data flow in HLLs (e.g., switch case or if-else), the assembly language must support conditional and unconditional *go to*. In x86 parlance, it is called a *jump* instruction. The generic form of jump is the jump instruction followed by a label. The **label** is a variable name that we give to an assembly instruction. This is needed because if you want to say (go to this instruction), how can you define “this” instruction? The label takes the following form: label: instruction. For example:

```
part: movq %rax, %rbx
. . . .
jmp part
```

In this example, “part” is the label for the move instruction. The “jmp” is a nonconditional jump; that is, it is always executed. There are some for conditional branches too. Let’s look at one of them.

For the jump if equal, or je, label, this instruction means jump to label if the zero flag is set to 1. In the programmer’s view of the assembly program, there are some flags that give information on the previous instruction (refer to Figure 5.18). One of the flags is called the zero flag, and it is set to 1 if the previous instruction has generated 0 as a result. For example, subtracting two registers and getting the result of 0.

Procedure Call Operations

One last item is related to procedure call—x86 has two instructions: CALL and RET to implement. However, the situation is more complicated than this. In HLLs, you have the concept of local variables and global variables. How is this enforced in assembly? Remember, the assembly program generated must behave in the same way you intended when you wrote the HLL program.

Assembly uses the concept of a stack, the very well-known data structure that works as last-in-first-out, to simulate the notion of local variables, passing arguments to procedure, and saving some of the registers in memory during a procedure call. Why do we need to save some registers? Because we have only 16 registers in x86, and programmers use way more variables than that in their HLL.

Vector Instructions

You may have realized that we have not mentioned floating points at all in the x86 operations. This is because there is another set of instructions and another set of registers for floating points—the **vector instructions**. These instructions operate not on individual registers, but on a vector (i.e., a group of numbers). So, an addition operation can add 32 numbers to another 32 numbers at once. That is, the first number is added to the corresponding first number in the second vector, the second number to second number, and so forth. These operations are usually very efficient in many applications.

5.5 Memory Hierarchy

Learning Objectives

By the end of this section, you will be able to:

- Discuss various memory and storage tools
- Differentiate between various types of storage technologies
- Explain how locality is used to optimize programs

For the processor to do its job, which is doing the calculations, it must be fed instructions and data. This means the overall performance depends on both the calculation's speed and the speed by which data and instructions are received. No matter how fast your processor is, you do not get good performance if the stream of instructions and data is not fast enough. Everything worked well in the early days of computing, from the 1940s until the early 1990s, and then computing hit a wall—a memory wall.

Researchers in industry and academia achieved good leaps in performance for processors by innovating ways to use transistors provided to them by Moore's law. They used those transistors, which were doubling per chip on average every 18 months, to add more features to the processor leading to better processor performance. However, the same was not done with memory, which resulted in a speed up gap between memory and processor. The gap started small and then got wider until it became a bottleneck of performance. Figure 5.19 shows the trend in the processor-memory performance gap.

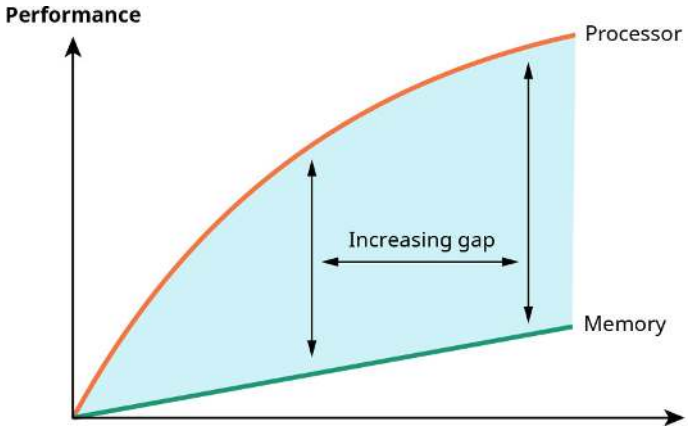


Figure 5.19 The gap between processor speed and memory speed is increasing. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

In this section, we explore memory technology. We will discuss the different technologies by which memory is built, explain what we mean by memory hierarchy, and see what researchers have done to deal with the processor-memory gap.

Memory and Storage

What is our wish list for the perfect memory? Probably speed—we want a fast memory. But be careful—memory speed is different from memory capacity. Memory capacity has increased throughout the years at a much faster rate than memory speed. We also want infinite capacity and persistence; that is, when the power is off, we want the memory to keep its content.

Next, we want to be able to pack a large amount of storage in as small an area as possible via density, which comes in handy especially for portable devices such as your smartwatch or smartphone. And what about the cost? If the memory is very expensive, the whole computer system is very expensive which means that nobody buys it; designers then have to put a smaller memory size into the computer system to keep the price low. But smaller memory means less functionality to the computer system and lower overall performance.

The reality is much less ideal. There is no single technology that excels in all these aspects. Some technologies are fast but more expensive, volatile, and less dense, while others are cheap and persistent but are relatively slower than other technologies. If we pick only one technology, we end up with a non-functional system. For example, if we pick the fast and volatile but expensive technology, the resulting computing system, which is probably very expensive, needs to be powered on indefinitely in order to retain the data. If we pick the persistent and cheap but slow technology, the system may be unusable due to its slow speed.

The word *storage* is usually used with persistence (long-term storage) while the word *memory* is used for volatility (short-term storage) even though this distinction may be blurred in future technologies. How can we get the best of both worlds?

CONCEPTS IN PRACTICE

The More You Know

Knowing about hardware is always beneficial to a software programmer. The cache, for example, is transparent to the programmer; however, if the programmer knows about the cache and how it works, they can write code that exhibits locality and gets good performance.

If you are writing a program that accesses a matrix, and you know how the matrix is stored in memory, you can adjust your code to access the matrix row by row (or column by column) to increase the locality which makes your program much faster.

The Memory Hierarchy

We have five items on our wish list for an ideal memory and, since there is no single technology that excels in all five, we must combine several technologies to come close to the ideal memory system. This ideal memory system must be fast, dense, persistent, large in capacity, and inexpensive. The technologies that we currently use have the following characteristics:

- Technology 1: very fast but expensive, less dense, and volatile
- Technology 2: faster and denser, but volatile and moderately expensive
- Technology 3: persistent and inexpensive but slow
- Technology 4: persistent and very inexpensive but very slow

We need to get the best of all of them, and the best way to combine them must ensure that we have higher capacity from technologies 3 and 4 but make technologies 1 and 2 closer to the processor so that they can respond faster to the processor. The obvious way to do this is to use an arrangement of storage available on a computer system in the form of a triangle as shown in Figure 5.20. We call this design **memory hierarchy**.

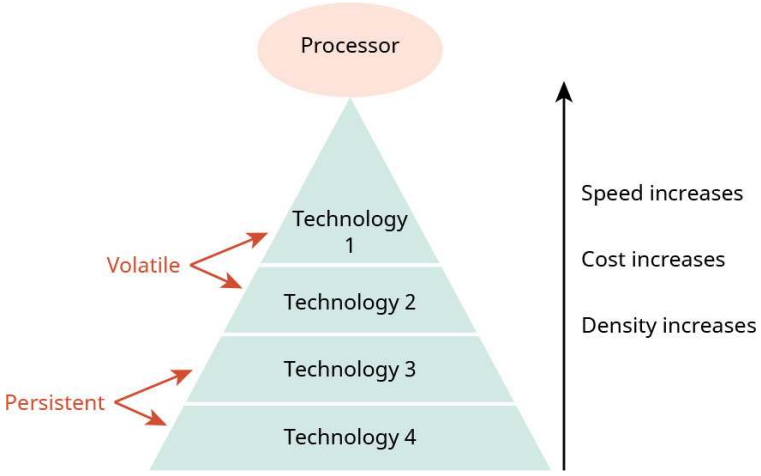


Figure 5.20 Memory hierarchy makes the best use of all technologies. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Table 5.3 gives the names of the technologies that we discuss later in this section. The processor is connected to the first cache using a set of wires called the bus. If the processor does not find what it wants, it goes to the next level, and so on.

	Technology Name	Use Case
Technology 1	Static random access memory (SRAM)	Cache memory • Volatile • Very fast • Expensive • Small in size (from few KB to few MB)
Technology 2	Dynamic random access memory (DRAM)	Main memory • Volatile • Fast • Less expensive • Average in size (1GB to 128GB)

Table 5.3 Technologies Used for Storage in a Typical Computer System

	Technology Name	Use Case
Technology 3	Solid-state drive (SSD)	Storage • Persistent • Slow • Cheap • Big in size (few GB to few TB)
Technology 4	Hard disk drive (HDD)	Storage • Persistent • Very slow • Very cheap • Several TB of storage

Table 5.3 Technologies Used for Storage in a Typical Computer System

Memory Technologies

Memory is volatile, at least for now, with research exploring other technologies for persistent memories, speed, storage, and expense. This covers technologies 1 and 2 and, therefore, they are closer to the processor. Technologies 1 and 2 cover two types of memories that both include **random access memory (RAM)**, which allows the processor to access any part of the memory in any order. Technology 2 is called DRAM, and technology 1 is called SRAM. Let us explore each one in turn.

Memory: DRAM

As you now know, the memory stores instructions and data, which are presented as 1s and 0s. One type of memory, **dynamic random access memory (DRAM)**, consists of a large number of capacitors. A **capacitor** is a very small electrical component that stores an electrical charge. A capacitor can be in one of two states: either it holds a charge, in which case we say that a 1 is stored in this capacitor, or it does not hold a charge, which means a 0. With millions of capacitors, we can store a large number of 1s and 0s. This is what you find in the specs of your laptop; when you say that you have 32GB of RAM, it means there are about 32 billion bytes in memory. Each byte consists of 8 bits. Each bit requires a capacitor.

Capacitors have a not-so-great characteristic though. When a charge is left on a capacitor for some time, the capacitor starts discharging and loses its charge. This means we lose the data stored in memory. Because of this, there is circuitry built inside the DRAM that, every few milliseconds, checks the capacitors and adds a charge to them. We call this the **refresh cycle**. It is done dynamically, hence the name dynamic RAM or DRAM.

The capacitors are not standalone by themselves. Transistors are used with them to help organize those capacitors into rows (also called word lines) and columns (also called bit lines) for addressing specific bits. Figure 5.21 shows a simplified view of DRAM. The cell, which stores 1 bit, is made up of the capacitor and some transistors.

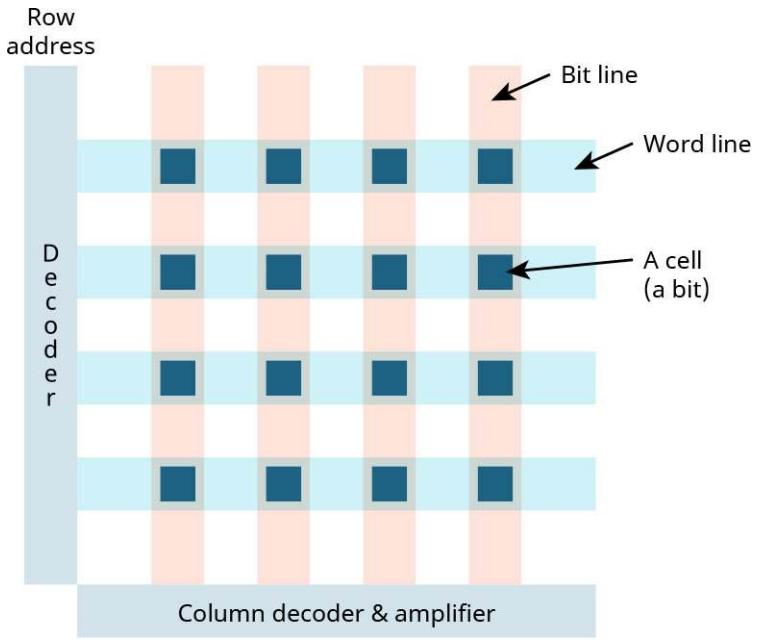


Figure 5.21 This simplified view of DRAM shows one bank. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

What is shown in Figure 5.21 is called a bank. Every few banks form a chip. Every few chips form a rank. A small memory board that contains several memory banks is a **dual in-line memory module (DIMM)**. Several DIMMs form a channel. This organization is shown in Figure 5.22. The reason for having this organization is twofold. First, if we have one huge 2-D array of memory cells, it is too complex, slow, and power hungry, so dividing it into parts makes those parts simpler and that means faster and less power hungry. Second, if there are several memory addresses that need to be accessed and they fall into different banks, for example, the memory can respond in parallel.



Figure 5.22 DRAM memory banks can be organized into chips, ranks, DIMMs, and channels. (credit: modification of "168 pin and 184 pin DIMM memory modules" by Veeblefetzer/Wikimedia Commons, CC BY 4.0)

You must have heard the term 64-bit machines, right? Most of our computer systems nowadays are 64 bit. One of the definitions of this term is that the connection between the processor and the memory has 64-bit width; its memory can send the data to, or receive data from, the processor in chunks of 64 bits.

The curve that we saw in Figure 5.19 shows the slow speed increase of the DRAM, which affects the performance of the overall system. If the processor must go to the memory for every instruction and every piece of data, the overall system performance is very low; therefore, computer designers speed things up by using a faster technology together with the DRAM. This faster technology, technology 1 in Figure 5.20 is called the SRAM.

Memory: SRAM

There are several reasons for the slow speed of getting the data from DRAM to processor. One is the much slower speed of DRAM technology relative to the processor. The second reason is that going off the chip that contains the processor and to the bus to reach the DRAM memory is a slow process. To overcome this, we need to have a faster memory technology inside the chip together with the processor.

The solution is to use **static random access memory (SRAM)** which keeps data in the computer's memory for as long as the machine is powered on. This means it does not need a refresh like the DRAM and is designed with a faster technology than DRAM. However, SRAMs are bigger in area; a single bit requires a large area in the chip as it needs four to six transistors, which is much larger than the capacitor in DRAM. Moreover, inside the chip we do not have a lot of space due to the existence of the processor itself. So, SRAM is a small, fast memory inside the chip that is connected to the processor from one side and to the DRAM off-chip from the other side, as shown in Figure 5.23. The DRAM is in the range of 8–64GB, while the SRAM starts from the KB range to a few MBs.

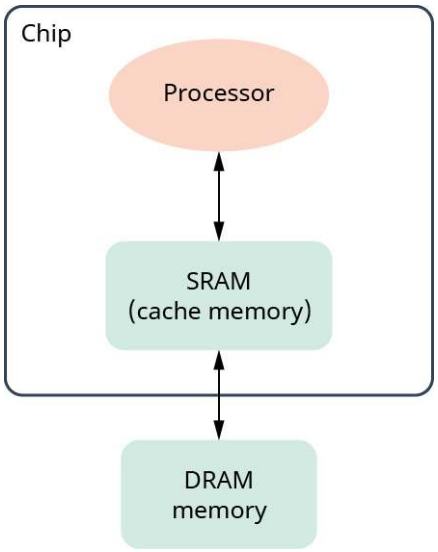


Figure 5.23 SRAM was introduced to overcome the slow speed of DRAM. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

One important distinction between the cache memory and the DRAM is that the former is programmer-transparent (i.e., its use cannot be managed by the programmer), but the latter is not. Your laptop has 32GB of RAM, which is the size of the DRAM, but you may not know how much cache your processor has. However, if you know how the cache works, you can write more efficient programs as we learn when we talk about locality.

In Figure 5.23, SRAM is more commonly referred to as **cache memory**, which is the memory that allows for high-speed retrieval of data. Let us see how the cache and the DRAM memory work together. From now on, whenever we say cache we mean the SRAM, and whenever we say memory, we mean the DRAM.

Suppose the processor executes a program that accesses array A consecutively. The processor starts with A[0] and asks the cache memory whether it has A[0]. Initially the cache is empty, so it does not have the needed data which is called a **cache miss**. The cache then gets the data from the memory. But instead of just getting A[0], it gets A[0], A[1], A[2], ..., A[x]. The number of extra elements the cache brings depends on the design of the cache. In most processors available now, the cache usually brings 64 bytes from the memory. So, if array A is an array of integers, that is, each element is 4 bytes in length, then the cache brings 16 elements from the memory, from A[0] to A[15]. The processor gets the A[0] it wants, and the extra elements brought from memory, which are in the cache. Now, if the processor wants A[1], it finds it right away in the cache, which is called a **cache hit**. However, if the processor instead needs A[17], then we have another cache miss and the cache gets to the memory again to bring several elements, including A[17].

Both the SRAM and DRAM, or cache and memory, are volatile—whenever there is a power perturbation or the machine runs out of battery, everything in the cache and the memory is gone. And we cannot build a full-fledged computer with volatile memory only; we need persistent storage too.

Storage Technologies

Storage exists in computer systems to ensure that data continues to exist even after the computer is powered off. Storage, presented as technologies 3 and 4 in Figure 5.20, has few characteristics that differ from DRAM and SRAM (technologies 1 and 2). The first, and most important one, is that they are persistent—they are non-volatile. The second characteristic is that they are slower in speed than DRAM and SRAM but have lower costs and higher capacities. It is to be noted that technologies 3 and 4 do not have to exist together in a computer system; you can have a computer with either or both.

Besides the storage that exists inside the computer system, there is a lot of storage in the cloud. That is, storage does not exist in your computer, but you can access it through the Internet. This storage is managed by big tech companies. For example, we have Azure from Microsoft, AWS from Amazon, Google Drive from Google, and so on. These companies are serving millions of users and are isolating users' data from each other. There are techniques to make each user access cloud storage and even software in the cloud.

Now, it is time to give them names. Technology 3 is called a solid state disk (SSD) and technology 4 is called hard disk drive (HDD). What are the differences and where does the commonly encountered term "flash drive" fit in? Let us start with the older technology first.

Hard Disk

Hard disks were the main storage solutions for all computers in the 1980s, 1990s, and until the mid-2000s. A **hard disk drive (HDD)** stores data on a rotating platter, has a very large capacity, and uses a small motor to rotate platters to get the data. You can easily buy an 8TB disk for a modest amount of money. The industry took about 25 years to move from 5MB disk to 1TB, and only two years to go from 1TB to 2TB, and, after that, the capacity increased by a whopping 60% per year. So, we have a very ample size with a low price but also a very slow disk. It is several orders of magnitude slower than the DRAM memory. The reason is shown in Figure 5.24. The main reason the disk is slow is due to the mechanical movement. Therefore, computer designers have been looking for storage that does not need mechanical movement drives and that does not have any moving parts.

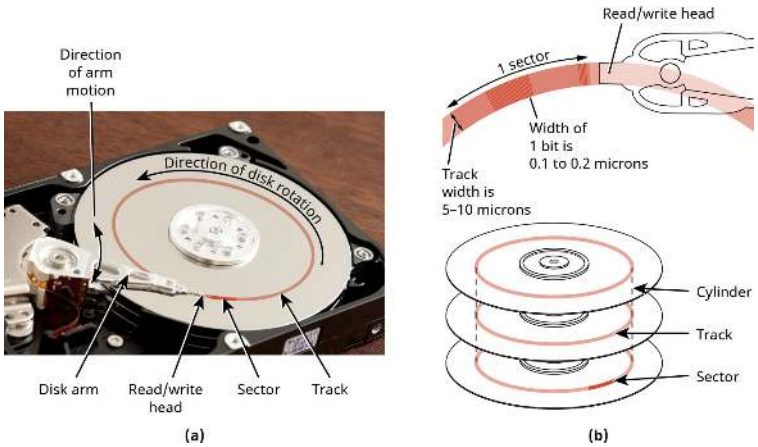


Figure 5.24 (a) The full hard disk drive (HDD) is (b) divided into platters, and each platter is divided into tracks and sectors. (credit a: modification of "Open HDD" by Gratuit/Freeimageslive, CC BY 3.0; credit b: attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Solid-State Drive

The term *solid state* means that a system does not have moving parts and is expected to be fast. A **solid-state drive (SSD)** stores data on a chip and is two to three orders of magnitude faster than HDD. SSD has two main

parts: the storage itself and the circuitry that accesses the storage. Nowadays SSDs use a storage technology called **flash memory**, which is a type of nonvolatile storage that can be electronically erased and reprogrammed. It is what you use in your thumb drives, just a bit faster. Flash memory in SSDs is based on NAND gates. A **NAND gate** is a type of logic gate used to store bits in flash memory. Flash memory is organized as pages, and a group of pages is called a **block**.

The circuitry that controls the flash memory, called the **translation layer**, has an important function. It maps the addresses to pages. One of the disadvantages of storage cells that make a page is that they wear out after 100 thousand to 1 million accesses. So, the translation layer tries to change the mapping to ensure that accesses are equally distributed among different cells. This is a complicated process and is one of the reasons SSDs are more expensive than HDDs. The sequential read from SSD reaches 7000 MB/s and the sequential write reaches 5000 MB/s. Random access to SSD has lower speed for reads and writes.

LINK TO LEARNING

Most laptops now have SSD storage, so it is good to know how SSD really works. Watch this video for a succinct explanation of how SSD works (https://openstax.org/r/76SSDHowItWorks) in the context of smartphones.

More About Cache Memory

As you have learned, cache memory is used as a fast, small memory inside the processor to close the gap between the processor speed and the memory speed. Given that latency is the rate of data transfer, assume DRAM memory's access latency is M cycles and the cache access latency is m. Also assume that for a specific program the probability of cache hits = p. Then the average latency of the combined cache + Memory = $mp + (1 - p)(M + m) = \boldsymbol{m + (1 - p)M}$.

Remember that whenever there is a cache miss, we have already spent m cycles searching the cache, then we go to the memory, which takes another M cycles. If we look at the equation $m + (1 - p)M$, we see that to get good performance, we need to do one or more of the following things:

- Have a faster cache and lower m.
- Increase cache hit rate and reduce p.
- Have a faster memory and lower M.

To have a faster cache, we must make it smaller in capacity, but smaller cache decreases **hit rate**, which is the number, usually a percentage, of times the cache was used to retrieve data. We cannot easily have faster memory to reduce M so the solution is to have more than one level of caches. Level 1 (L1), closest to the processor, is small in capacity and very fast but with a potentially low hit rate. When there is L1 cache miss, instead of going to the memory off-chip, we go to L2 cache, which is still on chip and bigger in capacity than L1. Most processors now have up to three levels of caches, as shown in Figure 5.25.

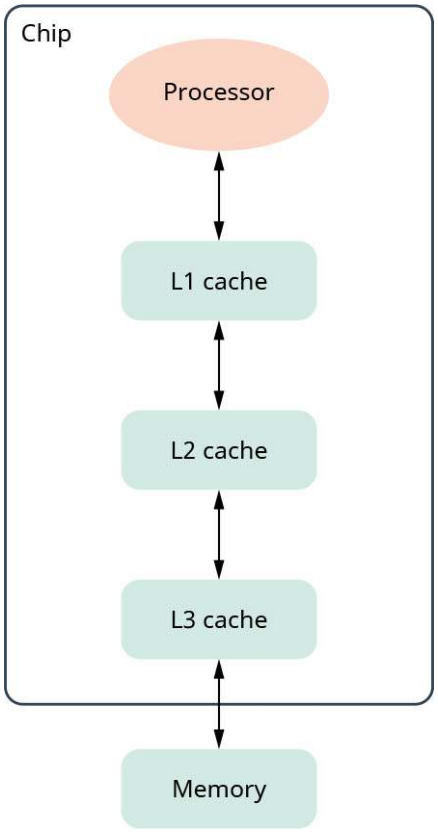


Figure 5.25 Most processors now have three levels of cache memory. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

Locality

Throughout our discussion of the memory hierarchy, we have determined the following:

- Memory is read in a chunk of consecutive 64 bits which is 8 bytes.
- Whenever there is a cache miss, the cache works to bring a cache block, not the few bytes the processor needs. Most caches now have a cache block of 64 bytes. Those 64 bytes are consecutive bytes brought from the lower-level cache, that come from the even lower cache (i.e., from L3 to L2 to L1). The L3 cache brought the block from memory.

When a processor repeatedly visits the same memory locales, it has **locality**. We can surmise that if a program accesses the data in a consecutive manner, it gets better performance, called **spatial locality**. Also, if we reuse the data as much as possible, we get better performance because the data is available in the cache, and we can increase the cache hits. We call this second criteria **temporal locality**.

We can get even better performance if the programmer writes efficient code that makes the best use of the underlying hardware. An efficient program has a very important characteristic called locality. For example, if you want to add two arrays together (i.e., *A*[0] + *B*[0], *A*[1] + *B*[1]), then you get a good performance if you access these two arrays sequentially from element 0 until the end of the arrays which is an example of spatial locality. So, whenever you are writing a program, pay close attention to how you access the data.

Instructions also reside in memory. If we have a for-loop that is executed several thousand times, the instructions in the loop body are reused in every iteration which is an example of temporal locality.

5.6 Processor Architectures

Learning Objectives

By the end of this section, you will be able to:

- Discuss the history and advancements in traditional processor architectures and computation models
- Define heterogeneity and discuss its effect on computer systems

Processors have a relatively short history that starts in the late 1960s; however, there have been big jumps since then. In this section, we learn about the evolution of processors from the dawn of processor design until today. The main measures of success of processors involve correctness, speed, power, reliability, and security.

Speed has been the main measure of success for some time. But then computer designers began to worry about battery life (for portable devices) and the electricity bill (for big machines) so power became a pivotal issue. As computers have invaded almost all aspects of our lives, reliability has become a must because we do not want computers to fail. With the widespread use of the Internet and peoples' need to be connected all the time, security has also become an issue.

Homogeneous Processor Architectures

The current processors contain several CPUs inside the chip. If these CPUs are copies of each other, the design is **homogeneous**. If there are different types of CPUs, some are fast but power hungry while others are slow but power efficient, the design is called **heterogeneous**. An example of heterogeneous processors is the chip inside the latest MacBook Pro.

In its earliest version, a processor was just a big, bulky, black box that got its instruction from memory, executed it, got the next instruction, and so on. What was bad about this simple design? First, having one big bulky circuit made it slow and power hungry. Second, not all instructions took the same amount of time; a floating-point computation took as much as ten times longer than an integer computation. But since this design was one box working with one clock, the clock cycle had to be as big as the slowest instruction. The processor executed at the speed of the slowest instruction; therefore, even though the design was simple, it suffered from performance and power issues. However, physics came to the rescue.

CONCEPTS IN PRACTICE

Social Media and Supercomputers

Billions of people use social media sites such as Facebook, X (formerly Twitter), YouTube, and Instagram every day and at the same time. This means we need supercomputers that can serve all these people, store and manipulate huge amounts of data in a short time, and not go down. This cannot be done with a simple multicore; it requires millions of multicores and thousands, if not millions, of accelerators such as GPUs, TPUs, and FPGAs.

Moore's Law and Dennard Scaling

In 1965, Gordon Moore (cofounder of Intel) published a short paper that predicted that the number of devices inside the processor would double every 18 months, called Moore's law. Since transistors are the building blocks of logic gates, and logic gates are the building blocks of pieces such as adders, multipliers, and registers, then more transistors would mean more features implemented in the processor, hopefully leading to better performance. More transistors inside the processor's chip meant that transistors would get smaller in size, and smaller transistors would mean faster transistors. Not only that, but Robert Dennard from IBM found that as transistors got smaller, the power consumed and dissipated was also reduced in a phenomenon known

as Dennard scaling.

Traditional Processor Architectures

Figure 5.26 shows how the processor evolved from the single cycle implementation to more sophisticated and higher performance designs thanks to Moore's law and its enabling technology, Dennard scaling.

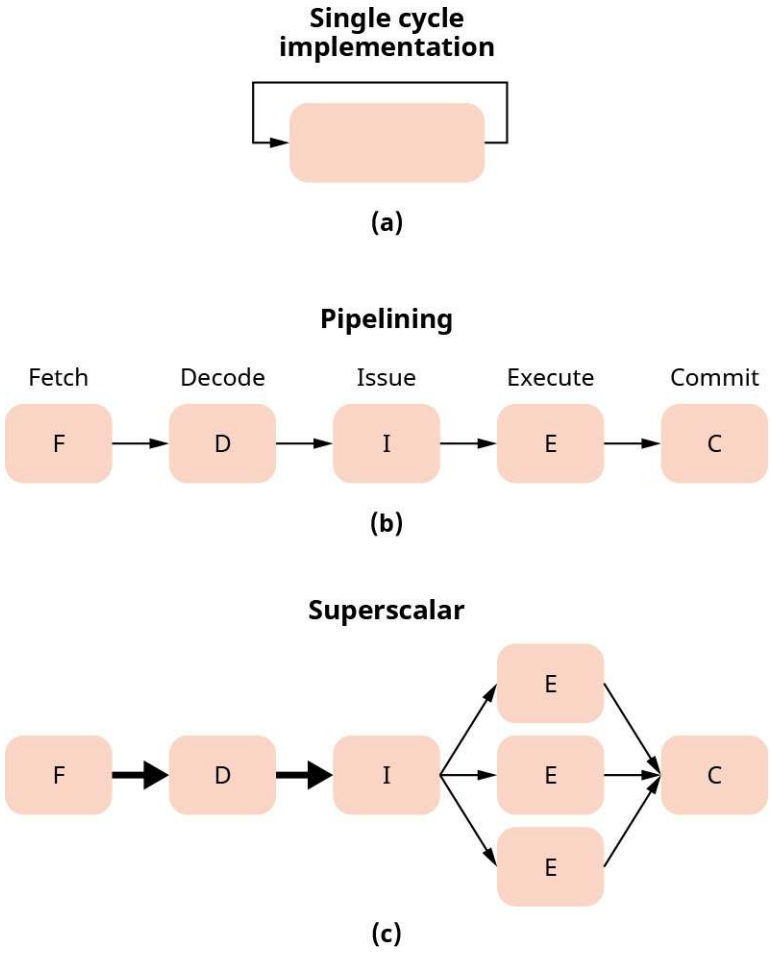


Figure 5.26 The processor has evolved from the (a) simple design single-cycle implementation to (b) pipelining to the very sophisticated (c) superscalar design in less than 70 years. (attribution: Copyright Rice University, OpenStax, under CC BY NC-SA 4.0 license)

If you think about what this bulky box in Figure 5.26(a) really does, you reach the following conclusion: it does few things repetitively with each instruction. It fetches an instruction from memory, decodes this instruction to know what needs to be done, and issues the instruction to the correct execution units (e.g., an integer operation to the integer execution unit and a floating-point operation to floating point execution unit).

After execution, it writes the result of the operation back to a destination specified by the instruction, which is called the commit. Given this description, why not take the bulky piece of hardware in Figure 5.26(a) and divide it into these pieces: fetch, decode, issue, execute, and commit? Each piece does the work needed by the following piece in a technique called **pipelining**, with each piece called a phase (Figure 5.26(b)). What do we gain from this?

First, each phase is now much less complicated, less power hungry, and faster. Second, once the fetch phase finishes fetching the first instruction and hands it to the decode phase, the fetch phase hardware is now free to fetch the second instruction. By the time the decode phase is done with instruction 1 and hands it to the issue phase, the fetch phase hands to it instruction 2 and starts fetching instruction 3, which is a form of parallelism.

If we take a snapshot of the pipeline during the execution of the program, we find different pipeline phases working on different instructions. We call this type of parallelism **temporal parallelism**. The benefit is that this parallelism—or better performance—is done with no involvement from the programmer. The hardware is doing it for you.

With more and more transistors available, designers move to executing several instructions at the same time, which means that several execution units are needed. The other phases must be modified to fetch/decode/issue several instructions at the same time, which is referred to as capability as shown in Figure 5.26(c). Another type of parallelism is **superscalar capability**, which is an execution unit that allows several instructions to be executed at the same time using another type of parallelism, **spatial parallelism**. If you look closely at Figure 5.26(c), you realize it combines both the temporal parallelism (from pipelining) and spatial parallelism (from superscalar capability).

The next step in enhancing a processor's performance is to fetch instructions from more than one thread, and you can technically execute two or more programs on the same processor at the same time. This is called **simultaneous multithreading (SMT)**, which Intel calls hyperthreading technology.

This last enhancement was introduced in the early 2000s, but something happened around 2004 that pushed both the hardware and software communities to change gears. Dennard scaling stopped and designers kept increasing the frequency of processors to make them faster. This led to a complete stall because increasing frequency resulted in drastic increases in power consumption.

Multiple Cores

With the stop of Dennard scaling and the inability to increase the frequency of the processor, it was time for a drastic change. On one chip, instead of putting one processor, designers decided to put multiple processors called cores inside the same chip which started the multicore era. All the processors we use today, from our watches to big supercomputers, are multicore processors.

The trend is to increase the number of cores per chip while decreasing their frequency. Each core in the multicore is an SMT to avoid an increase in power consumption as well as an increase in chip temperature. There is one catch though; all previous techniques (such as pipelining, superscalar, and SMT) were giving us performance without the programmers getting involved. To make use of all the cores in the chip with multicore, programmers must write parallel code that requires the use of a parallel programming language. This is bound to become the norm.

LINK TO LEARNING

Microprocessors (also known as processors) have evolved in the last half century in many ways. Visit this site to learn more about microprocessor trend data (https://openstax.org/r/76CMicroproData) from 1970 to the present.

Heterogeneous Processor Architectures

Multicore processors are designed to be good on average for most applications. However, they do not give the best performance for every single program. Designers have started introducing chips that have excellent performance, better than multicore, but for a small subset of program types. Examples of these chips are graphics processing units (GPUs), field programmable gate arrays (FPGAs), and tensors processing units (TPUs). The idea is to start the program on a multicore until there is a part where other chips excel. In that case, the multicore sends that piece of code to the other chips, and this gives rise to parallel programming for heterogeneous systems (i.e., computer systems that have chips with different capabilities). Laptops can now have a multicore plus GPU.

Multiple Nodes

Having multicore plus accelerator chips (e.g., GPUs) on the same board is now the norm and is called a node. But what about big machines that run in the cloud to give us services such as Amazon, Facebook, and X (formerly Twitter)? These big machines are built using thousands, if not millions, of nodes and they need an even more sophisticated way of programming.

Chapter Review

Key Terms

abstraction removal of unimportant elements of a program or computer code that distract from its process
addressing mode location in the memory that contains the required data
American Standard Code for Information Interchange (ASCII) unit code or unique binary number
arithmetic logic unit (ALU) piece of hardware inside the CPU that performs computations and logical operations such as comparisons
assembler takes the assembly program that takes I/O that contains the equivalent of an assembly program
assembly language low-level language that is designed to be computer friendly
bit binary digit made up of 1 or 0
block group of pages in flash memory
bus data pathway
byte 8 bits
cache hit data found in the cache needed by the processor
cache memory memory that allows for high-speed retrieval of data
cache miss when the cache is empty so it does not have the needed data
capacitor very small electrical component that stores an electric charge
complex instruction set computer (CISC) complex architecture structure that assists in executing such operations as mathematical computations and memory storage
disk storage mechanism for data
dual in-line memory module (DIMM) small memory board that contains several memory banks
dynamic library collection of libraries that may be linked during execution or while the program is running
dynamic random access memory (DRAM) consists of a large number of capacitors
executable program stored inside a computer
flag tells a program if a condition has been met
flash memory type of nonvolatile storage that can be electronically erased and reprogrammed is called
floating point number (also, real number) one with a decimal point in the middle
hard disk drive (HDD) stores data on rotating platter, has a very large capacity, and uses a small motor to rotate platters to get the data
heap stores dynamically allocated data
heterogeneous design feature where CPUs are different in speed, power, or efficiency
high-level language (HLL) most evolved method by which a human can direct a computer on how to perform tasks and applications
hit rate number, usually a percentage, of times the cache was used to retrieve data
homogeneous when CPUs are copies of each other in design
IEEE 754 standard format used by computers that support floating points with very few exceptions
input/output (I/O) interface that helps the CPU talk to other I/O devices such as a keyboard or mouse
instruction set architecture (ISA) set of instructions recognized by each processor family
label variable name that we give to an assembly instruction
library collection of files, functions, or scripts that are cited within a program's code
loader puts the executable into memory and arranges its content in a specific way to make it ready for execution by the processor
locality when a processor repeatedly visits the same memory locales
logic gate main building block that forms the processor
memory hierarchy arrangement of storage available on a computer system usually in the form of a triangle
microarchitecture architecture, or design, of the processor or microprocessor
NAND gate type of logic gate used to store bits in flash memory
neuromorphic computer nontraditional computer built to act like a simplified version of the brain

object code designates output of the assembler
one's complement obtained by flipping each 1 in the original binary number to 0 and each 0 to 1
operand value used as an input for an operator
operating system (OS) only piece of software that can directly access the hardware
pipelining technique where each piece of the process does the work needed by the following piece
processor another name for the CPU
program counter (PC) register that keeps track of instructions to be executed
pseudo-assembly assembly instruction that does not have a counterpart in the machine code
random access memory (RAM) allows the process to access any part of the memory in any order
reduced instruction set computer (RISC) simple instructions
refresh cycle regular operation that DRAM memory does by adding charges to memory cells in order to lose the data stored as charges
register memory unit that functions at a very high speed
register file group of registers
scheduling when the OS manages which program to use with what part of the hardware at any given time
signed integer integer that can be negative or positive (sign of + or –)
simultaneous multithreading (SMT) when two or more programs execute on the processor at the same time
solid-state drive (SSD) stores data on a chip and is two to three orders of magnitude faster than HDD
spatial locality idea that if a program accesses data in a consecutive manner, it gets better performance
spatial parallelism type of parallelism capability
stack used to store local variables
static random access memory (SRAM) keeps data in the computer's memory as long as the machine is powered on
superscalar capability execution unit that allows several instructions to be executed at the same time using spatial parallelism
temporal locality reuses cached data to get better performance
temporal parallelism allows for different pipeline phases to work on different instructions
transistor lets an electric current pass (on state) or blocks it (off state)
translation layer circuitry that controls the flash memory
two's complement one's complement with 1 added
unsigned integer non-negative integer that starts from 0
vector instructions another set of instructions and another set of registers for floating points
virtual memory technique the OS uses to isolate different programs from each other so they do not overwrite each other's data or corrupt each other's files

Summary

5.1 Computer Systems Organization

- The main components of a computer system include the processor, the main memory, the disk, and I/O devices.
- The components interact to execute computer programs efficiently.
- Applications are written in high-level languages that use a program to translate the language into the machine-level programs that computers understand.
- This computer system organization is mostly the same in your tablet or smartphone as it is in the huge systems running services like Facebook or Google. They just differ as it comes to how powerful each piece is, how many CPUs are there, the size of the memory and disk, and so on.

5.2 Computer Levels of Abstraction

- The levels from problem definition to assembly language are related to the computer science field. People studying computer science explore how to solve a problem using an algorithm, how to translate this

algorithm into a programming language, then how to translate this programming language into a language that computers can understand.

- Computer processors operate at various levels of abstraction going from the digital logic level up to the microarchitecture level and the machine language level. The highest level of abstraction for application programs is that obtained by writing programs in a high-level language. Using a compiler makes it possible to generate a representation of such programs at a lower level of abstraction known as assembly language. That representation uses instructions that are part of the instruction set architectures (ISA) specific to the processor family in use.
- The operating system (OS) is the only piece of software that can directly access the hardware of a computer. All other programs must interface with the OS to have it achieve a specific task in a secure fashion by scheduling the corresponding process. The OS stores data and programs on disk in an organized way, using a file system, and allocates memory to programs.
- Upcoming computer designs call for new computing abstractions that will deviate from the traditional binary logic. For example, quantum computing uses qubits and neuromorphic computing uses hardware neurons.

5.3 Machine-Level Information Representation

- The most frequently used data items are stored inside a computer.
- An integer is a number that does not have a floating point; 7 is an integer but 3.14 is not. Integers are divided into two categories: signed and unsigned. Signed integers can be positive, negative, or 0. Unsigned integers are 0 or positive; therefore, they do not need a sign because they will never be negative. This is why we call them unsigned.
- Real numbers, known as floating point numbers, are the numbers that represent fractions. Integers cannot represent numbers like 3.14 or –1.25. This is the role of floating point numbers.
- Characters and symbols on your keyboard are represented inside the computer as 1s and 0s. Every character has its own code.
- The range of numbers that *n*-bit binary number can present if interpreted as a signed or an unsigned integer is helpful to you when you write a program because if you know the values that a variable in your program may take, you can make a precise decision about the type of that variable when you declare it.

5.4 Machine-Level Program Representation

- An x86-64 Intel processor uses a complex instruction set computer architecture (CISC). This type of architecture combines many simple instructions into a simple complex one. Other processor architectures use a reduced instruction set architecture (RISC) based on simpler instructions. RISC V is a relatively new RISC ISA that is getting popular than CISC. All new high-tech companies use RISC instruction sets in their hardware. For example, ARM assembly, which is a RISC assembly, is used in most portable devices.
- Assembly language makes use of processor instructions that are part of the instruction set architecture (ISA) of the processor being used. These instructions can be converted to binary code, referred to as machine language code, using an assembler.
- Assembly language provides specific instruction to perform common operations such as addition and multiplication of signed and unsigned integers.

5.5 Memory Hierarchy

- We must use different technologies to build a near ideal memory because each technology has some good characteristics and a few shortcomings. Therefore, we use multiple technologies to get the best of all. Various types of memory, such as DRAM and SRAM, are available and differ in access speed and associated costs.
- Those technologies are organized as a hierarchy. Technologies that are fast but expensive are at the top of the hierarchy, but their storage capacity is not big. As we go down the hierarchy, the technology is slower, but cheaper; hence, we use a lot of storage from it.
- That is, characteristics of each technology determine its place in the hierarchy.

- By paying attention to locality in accessing the data, the programmer can get the best performance from the hierarchy.

5.6 Processor Architectures

- Processors evolved from a single cycle implementation to multicore.
- The norm now is to have a multicore processor working in tandem with one or more accelerator chips.
- The norm in software development is to write parallel code for such a heterogeneous system. Sequential programming will soon be dated.

Review Questions

1. What is the difference between CPU and ALU?
2. What is the difference between the processor used in your smartphone and the one used in your desktop, tablet, or laptop computer?
3. What is the definition of abstraction?
   a. a piece of software that takes a program written in a given HLL and generates another program
   b. the most evolved method by which a human can direct a computer on how to perform tasks and applications
   c. the set of instructions recognized by each processor family
   d. removal of unimportant elements of a program or computer code that distract from its process
4. What is the term for the technique the OS uses to isolate different programs from each other so they do not overwrite each other's data or corrupt each other's files?
   a. virtual memory
   b. scheduling
   c. neuromorphic computer
   d. assembler
5. Why do computers only work with 1s and 0s?
6. Why is the OS the only software allowed to access the hardware?
7. Do a big supercomputer and small budget laptop use the same levels of abstraction?
8. What type of number would we need to represent the number of marbles in a bag?
   a. signed integer
   b. unsigned integer
   c. floating point number
   d. Boolean number
9. What type of number would we need to represent the temperature outside to the nearest whole number?
   a. signed integer
   b. unsigned integer
   c. floating point number
   d. Boolean number
10. What type of number would we need to represent the amount of money in a bank account?
    a. signed integer
    b. unsigned integer
    c. floating point number
    d. Boolean number
11. What is the two's complement of the binary number 11100101?
    a. 01100101

b. 11100100
c. 00011010
d. 00011011

12. What is the one's complement of the binary number 10100010?
a. 00100010
b. 10100011
c. 01011101
d. 01011110

13. If you see a series of bits, can you know whether they present a signed or an unsigned integer, a floating point, or a character?

14. Why do we need unsigned integers?

15. What is the purpose of a linker?
a. A linker puts the executable into memory and arranges its content in a specific way to make it ready for execution by the processor.
b. A linker tells a program if a condition has been met.
c. A linker takes all the generated object files, looks for needed libraries, and then links everything together in one executable file.
d. A linker is a register that keeps track of instructions to be executed.

16. What is the output of the assembler called?
a. object code
b. source code
c. dynamic library
d. static library

17. What stores dynamically allocated data?
a. stack
b. register
c. bus
d. heap

18. Why do you need to learn assembly programming?

19. What is the main philosophy behind CISC?

20. What is the main philosophy behind RISC?

21. What technology do we use to build a cache?
a. static random access memory (SRAM)
b. NAND gate
c. dynamic random access memory (DRAM)
d. solid state drive (SSD)

22. What technology do we use to build memory?
a. static random-access memory (SRAM)
b. NAND bate
c. dynamic random-access memory (DRAM)
d. solid state drive (SSD)

23. What term means that, when the cache is empty, it does not have the needed data?
a. cache hit
b. cache memory

c. cache miss
d. cache reset

24. Why is it necessary to have multiple levels of cache memory?
a. to make it harder for the CPU to obtain instructions from RAM
b. to make it faster for the CPU to obtain data from RAM
c. just in case one level of cache fails
d. to be able to execute instructions in parallel

25. Is programming a single core or multicore easier? Why?

26. Why did the industry move to multicore?

27. What is the term used to describe when two or more programs execute on the processor at the same time?
a. temporal parallelism
b. spatial parallelism
c. simultaneous multithreading (SMT)
d. superscalar capability

Conceptual Questions

1. Why is the memory connected directly to the CPU?

2. Both the disk and memory store programs and data. Why do we need them both in a computer system?

3. Why do we need an assembler? Why don't we make compilers generate machine language directly?

4. If there are two processors that understand the same ISA, does this mean they have exactly the same microarchitecture?

5. Can the step from algorithms to HLL programs be automated instead of being done by a human being? Justify your answer.

6. The idea of compilers and assemblers made HLL more portable. Why is that?

7. Suppose we have a list of 1,000 numbers ordered in ascending order. We need to find whether a specific number is in the list or not. What is the best algorithm to accomplish this?
a. Scan the list from first number to last number until you find the number you want or reach the last number.
b. Scan the list from last number to first number until you find the number you want or reach the first number.
c. Go to the middle of the list and see whether the number you are looking for is bigger or smaller than the middle number. If it is bigger, discard the lower half. If it is smaller, discard the higher half. Then redo the same in the smaller list.
d. Take a quick look at the list and decide whether the number is present.

8. Why is the two's complement a good choice for presenting signed numbers?

9. As a programmer, do you think it is useful to know about data presentation? Why?

10. Why are computers slower in dealing with floating points than integers?

11. For portable devices, such as your smartphone, do you think processors supporting CISC ISA or RISC ISCA should be used? Why?

12. Is it possible to build a de-compiler? That is, if given an assembly code, can we bring the original HLL code? Explain.

13. Why are the different technologies organized as a hierarchy?
14. What is the relationship between SSD and flash memory?
15. Do you think it is better to have more cores in the chip or to use multiple chips with fewer cores each? Why?
16. Why is heterogeneous computing here to stay?

Practice Exercises

1. Perform a search on the Internet about the specifications of your own computer as well as other similar computers from companies like Lenovo or Dell in terms of memory size and disk size. What is the ratio disk size to memory size in each computer and why?
2. Document how you can obtain the amount of cache memory on your computer.
3. Search the web for five different ISAs. For each one, find which companies are building processors that use that ISA. Finally, for each processor from these companies, check whether this processor is used on your portable device (smartwatch, smartphone, tablet), your laptop or computer, or in big supercomputers and data centers.
4. You are using the Internet to access a website of your choice. Create a diagram showing the various levels of abstraction of a computer system to explain this particular scenario.
5. Research and explain a few key responsibilities of operating systems.

Suppose you wish to express –64 as a two's complement number.

6. What is the minimum number of bits we will need?
7. With this minimum number of bits, what is the largest positive number you can represent, assuming signed numbers of course? (Answer in both decimal and binary.)
8. With that same number of bits you used in the previous questions, what is the largest unsigned number you can represent? (Answer in both decimal and binary.)
9. Represent the decimal digit 90 in binary.
10. Add the following binary numbers: 0111000101 + 0000100101?
11. Find the one's complement of 1010001000011111.
12. Find the two's complement of 000010001111110.
13. Make a list of at least three ISAs in existence today. For each one, find out whether it is CISC or RISC and in which processors it is used. Finally, take the list of these processors and see whether they are used in high-performance computing machines or portable devices.
14. Make a list of three or four accelerators and, for each one, describe the type of applications they excel at and why.

Problem Set A

1. A cache has an access time of 1 cycle. The computer with that cache experienced an average memory access time of 4 cycles, and the hit rate is 70%. What is the access time of the main memory? Did we benefit from having a cache in this system?
2. Research and explain a few key responsibilities of operating systems.

Given $X = 01100110$,

3. What is the value of X once you logically shift X to the right by two digits?
4. What is the value of X once you arithmetically shift X to the right by two digits?

Given $X = 10100110$,

5. What is the value of X once you logically shift X to the right by two digits?
6. What is the value of X once you arithmetically shift X to the right by two digits?
7. What is the value of X if you arithmetically shift it to the right by eight digits?

Perform the following number conversions:

8. hexadecimal value 40A5F916 to binary
9. binary value 11011010011010111010 to hexadecimal
10. Considering the arguments passed in, what does the following assembly language program do?

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

movq	%rdi, %rax
subq	%rsi, %rax
movq	%rsi, %rdx
subq	%rdi, %rdx
cmpq	%rsi, %rdi
cmovle	%rdx, %rax
ret	

11. Suppose that we have a system with memory access time of 8 cycles. We need to add to that system a cache with 2 cycles access time. What is the smallest hit rate needed to make that cache beneficial? (Hint: avg access time must always be an integer number not floating point.)
12. A cache has an access time of 1 cycle. The computer with that cache experienced an average memory access time of 4 cycles, and the hit rate is 70%. What is the access time of the main memory? Did we benefit from having a cache in this system?

Problem Set B

1. Suppose that we have a system with memory access time of 8 cycles. We need to add to that system a cache with 2 cycles access time. What is the smallest hit rate needed to make that cache beneficial?
2. Explain how there are multiple levels of abstraction in a computer and explain how at each level, complex implementation knowledge of the lower levels is not needed.
3. Calculate 0 1000 0001 110...0 plus 0 1000 0010 00110...0. Both are single-precision IEEE 754 representations.

4. We derived the equation $m + (1 - p)M$ for average access latency when we have one cache and memory. Extend this equation to include two-level cache followed by memory.

Thought Provokers

1. What are the desired characteristics of a good computer system, aside from speed?
2. When a company decides to design a new processor to compete in the market, what are the criteria that this company needs to tackle to be competitive? Be careful—speed is not enough.
3. Consider our startup company that is 100% committed to leveraging innovative technologies as a business growth facilitator. Describe at a high-level how it could combine the use of Unicode and Large Language Models to translate documents in any language. What would be some of the limitations of such a solution using existing technology?
4. With several levels of cache, the data in L1 is a subset of the data in L2 which is a subset of L3. This is called the inclusion property. This makes the data a bit redundant and loses chip area to store repetitive data. Do we gain anything from violating this inclusion property? If yes, what do we gain? What are the challenges? If no, why not?
5. Now that Moore's Law is coming to an end, how do you think the computer industry can get more performance from computer systems? Suggest several solutions.

Labs

1. Make a list of the I/O devices on your laptop, and another list of the software programs installed on your laptop. This second list is a bit tricky.
2. Find out the type of processor on your computer or laptop. Once you get the type, find out the following information about it:
   - Which company designed it?
   - How many transistors does it have?
   - What ISA does it use?
3. Using the link provided earlier in this section about translating from HLL to assembly, write a sample program and generate its assembly. Look at the assembly generate, understand it well, and then write yourself another version of this assembly that does the same thing.